

Universidade Federal de Minas Gerais

Automação em Tempo Real

Sistema de Inspeção Automática de Túneis

Relatório Final

Leandro Lopo Matrícula 2021020520
Rodrigo Martins Matrícula 2023421459

Professor: Leandro Freitas

Belo Horizonte
2026

Sumário

1	Introdução	1
1.1	Objetivos	1
2	Proposta de Arquitetura	1
2.1	Visão geral	1
2.2	Comunicação MQTT	3
2.3	Threads do núcleo	4
2.4	Sincronização	5
2.5	Periodicidade	6
3	Implementação	7
3.1	Controle de navegação	7
3.2	Posição e reconstrução	8
3.3	Detecção e câmera	8
3.4	Coleta de dados	8
3.5	Inicialização e encerramento	8
4	Resultados dos Testes	9
4.1	Ambiente	9
4.2	Matriz de testes	9
4.3	Evidências visuais	10
4.4	Limites da validação	12
5	Análise de Capacidade e Escalabilidade	12
5.1	Método de medição	12
5.2	Período mínimo e garantia temporal	12
5.3	Gargalos de escalabilidade	13
6	Link para o Vídeo	13
7	Conclusão	14
A	Código-fonte	15
A.1	Compilação	15
A.2	Inicialização dos processos	15
B	Cabeçalhos C++	17
B.1	buffers.hpp	17
B.2	log.hpp	18
B.3	mqtt_topics.hpp	18
B.4	shared_state.hpp	18
B.5	tasks.hpp	19
B.6	time_utils.hpp	19
B.7	types.hpp	20

C	Núcleo C++	20
C.1	main.cpp	20
C.2	comando_navegacao.cpp	23
C.3	controle_navegacao.cpp	24
C.4	distancia_percorrida.cpp	26
C.5	reconstrucao_superficie.cpp	27
C.6	coletor_dados.cpp	30
C.7	inspecao_camera.cpp	31
C.8	mqtt_sensor_subscriber.cpp	32
C.9	mqtt_command_subscriber.cpp	35
C.10	mqtt_actuator_publisher.cpp	38
C.11	mqtt_state_publisher.cpp	40
C.12	mqtt_surface_publisher.cpp	42
C.13	simulacao_sensores.cpp	44
C.14	log.cpp	45
C.15	time_utils.cpp	45
D	Simulação e Operação Remota	45
D.1	simulation_gui.py	45
D.2	simulation_physics.py	56
D.3	simulation_test.py	59
D.4	actuator_receiver_test.py	60
D.5	remote_gui.py	61
E	Benchmark Temporal	72
E.1	benchmark_temporal.cpp	72

Lista de Figuras

1	Ilustração própria do problema: inspeção do perfil do teto por um robô com LIDAR.	1
2	Arquitetura final distribuída em processos e comunicação MQTT.	2
3	Interface de simulação na região de falha: LIDAR em 128 cm, câmera ligada e velocidade reduzida.	10
4	Operação remota exibindo inspeção ativa e descontinuidade na superfície reconstruída.	11
5	Detecção determinística de falha e execução da tarefa de câmera.	11
6	Amostras gravadas em <code>surface_points.csv</code>	11

1 Introdução

O projeto implementa um sistema simulado de inspeção automática do teto de um túnel. Um robô móvel recebe comandos de aceleração, mede sua velocidade, detecta transições de encoder e lê a distância até o teto por LIDAR. O núcleo combina posição e distância para reconstruir o perfil do túnel. Uma variação superior ao limite configurado caracteriza uma possível falha, reduz a velocidade e aciona uma tarefa de inspeção por câmera.

A solução é formada por três programas: núcleo concorrente em C++, simulação gráfica em Python/Pygame e operação remota em Python/Pygame. O broker Mosquitto realiza a comunicação entre processos por MQTT. Internamente, o núcleo usa threads, filas produtor-consumidor, mutexes, variáveis de condição e tarefas periódicas, aplicando diretamente os conteúdos de programação concorrente, sincronização, entrada e saída assíncrona, IPC e sistemas de tempo real estudados na disciplina.

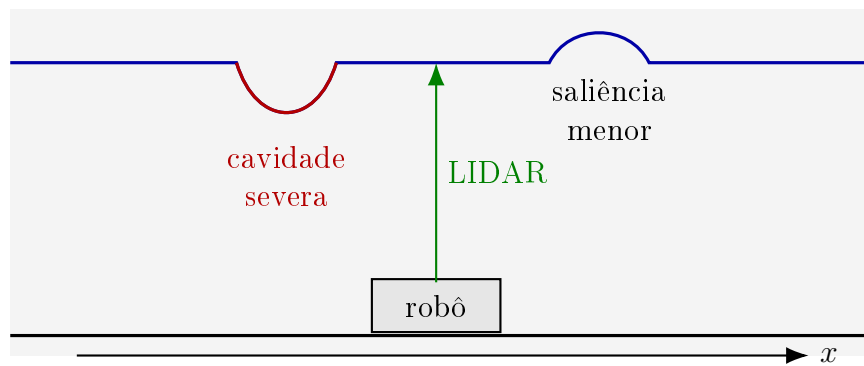


Figura 1: Ilustração própria do problema: inspeção do perfil do teto por um robô com LIDAR.

1.1 Objetivos

Os objetivos principais são:

- simular o movimento do robô, o encoder, o LIDAR e irregularidades do teto;
- executar controle automático PID e permitir comandos manuais remotos;
- reconstruir e registrar o perfil do túnel durante o percurso;
- detectar variações severas, acionar a câmera e reduzir a velocidade;
- distribuir tarefas concorrentes com acesso seguro aos dados compartilhados;
- integrar C++ e Python por comunicação MQTT assíncrona;
- avaliar funcionamento, capacidade e limitações temporais do protótipo.

2 Proposta de Arquitetura

2.1 Visão geral

A arquitetura final separa interface física simulada, processamento e supervisão. Essa separação evita acoplar a taxa de desenho das interfaces ao núcleo de controle e permite

iniciar cada processo de maneira independente.

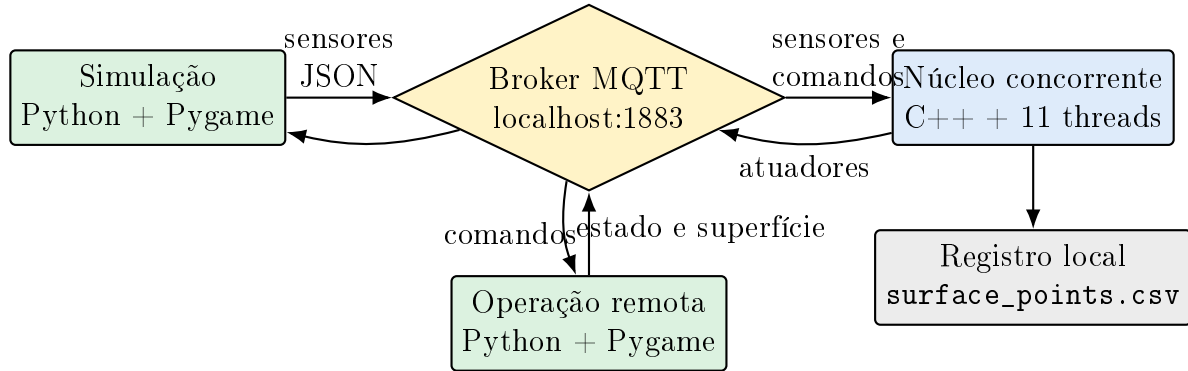


Figura 2: Arquitetura final distribuída em processos e comunicação MQTT.

Tabela 1: Processos e responsabilidades da arquitetura.

Processo	Tecnologia	Responsabilidade
Simulação	Python, Pygame e Paho MQTT	Integra dinâmica do robô a cada 80 ms, gera encoder/LIDAR com ruído, publica sensores e aplica os atuadores recebidos.
Núcleo	C++17, <code>std::thread</code> , libmosquitto e nlohmann/json	Executa controle, cálculo de posição, reconstrução, detecção, câmera, coleta e gateways MQTT.
Operação remota	Python, Pygame e Paho MQTT	Envia modo, direção, setpoint e limite de falha, além de exibir o estado e a superfície reconstruída.
Broker	Mosquitto	Desacopla publishers e subscribers usando tópicos MQTT.

A Tabela 1 evidencia a separação entre planta simulada, decisão de controle e supervisão. A **simulação** representa a parte física: integra aceleração e velocidade, movimentação do robô e produz os sinais que, em uma aplicação real, viriam dos sensores. Ela também recebe os comandos calculados pelo núcleo, fechando a malha de controle. O período de 80 ms foi mantido igual ao do controlador para que cada atualização física possa gerar uma nova amostra coerente.

O **núcleo C++** concentra as funções que exigem ordem, sincronização e resposta previsível. Como ele não desenha interfaces, dedica-se a receber dados, executar o controle, reconstruir a superfície e produzir atuadores. Essa separação impede que variações na taxa de quadros do Pygame atrasem diretamente o PID. A **operação remota** atua como supervisor, alterando parâmetros e apresentando resultados sem participar do cálculo de cada ciclo. Assim, o robô continua processando as últimas configurações válidas mesmo que a janela remota demore para redesenhar.

O **broker Mosquitto** elimina conexões diretas entre cada par de programas. Cada processo conhece apenas os tópicos que publica ou assina, o que facilita substituir a simulação por sensores reais ou executar a operação remota em outra máquina. O CSV

complementa essa arquitetura com uma saída persistente e independente da interface gráfica. Dessa maneira, os pontos já coletados permanecem disponíveis para análise mesmo depois que a janela é fechada.

2.2 Comunicação MQTT

O MQTT foi adotado como IPC porque os três programas são processos independentes e usam linguagens diferentes. O broker é executado em `localhost:1883`. O núcleo C++ se conecta por `libmosquitto`, enquanto as interfaces Python usam `Paho MQTT`. As mensagens são codificadas em JSON, formato diretamente suportado nas duas linguagens e legível durante a depuração. O protótipo usa QoS 0, suficiente para a execução local, mas sem confirmação de entrega.

Tabela 2: Tópicos MQTT implementados.

Tópico	Origem → destino	Período	Campos principais
<code>atr/sim/sensors</code>	Simulação → núcleo	80 ms	encoder, LIDAR, velocidade, posição real e timestamp
<code>atr/core/actuators</code>	Núcleo → simulação	80 ms	aceleração e câmera
<code>atr/remote/commands</code>	Operação → núcleo	por comando	modo, direção, parada, setpoint e limite
<code>atr/core/state</code>	Núcleo → operação	80 ms	modo, inspeção, posição, velocidade e atuadores
<code>atr/core/surface</code>	Núcleo → operação	por amostra	timestamp, posição, LIDAR filtrado e confiança

Os cinco tópicos da Tabela 2 formam dois fluxos principais. O primeiro corresponde à **malha robô-núcleo**. A cada ciclo, `atr/sim/sensors` entrega encoder, LIDAR e velocidade ao núcleo. Depois do processamento, `atr/core/actuators` devolve aceleração e estado da câmera. Como ambos são publicados a cada 80 ms, a simulação recebe comandos na mesma escala de tempo usada pelo controlador.

O segundo fluxo é de **supervisão**. `atr/remote/commands` só é publicado quando o operador altera modo, direção, parada, setpoint ou limite de falha, evitando tráfego periódico sem mudança. No sentido contrário, `atr/core/state` envia uma visão consolidada do robô a cada 80 ms, suficiente para atualizar os indicadores da interface. Já `atr/core/surface` é orientado a dados: cada ponto reconstruído é publicado assim que fica pronto, permitindo que o gráfico cresça durante o percurso.

Os campos de cada mensagem acompanham a função do tópico. O `timestamp` preserva a referência temporal, enquanto encoder e LIDAR alimentam a reconstrução. A velocidade fecha a realimentação do PID, e os campos de posição, confiança e inspeção atualizam a interface. As publicações usam QoS 0 e não são retidas. Essa escolha reduz a sobrecarga porque uma amostra antiga pode ser substituída pela seguinte, mas não oferece garantia

de entrega. Em uma aplicação crítica, seria necessário usar QoS 1, números de sequência e uma estratégia de reconexão.

O receptor valida e converte o JSON dentro de `try-catch`. Uma mensagem malformada é registrada e descartada sem terminar o processo. O campo `finalizado` encerra o pipeline e acorda consumidores ainda bloqueados.

```

1 try {
2     const json mensagem = json::parse(payload);
3     SensorData leitura;
4     leitura.i_encoder = mensagem.at("i_encoder").get<bool>();
5     leitura.i_lidar = mensagem.at("i_lidar").get<int>();
6     leitura.timestamp = mensagem.at("timestamp").get<double>();
7     // O mesmo evento alimenta os fluxos de LIDAR e encoder.
8 }
9 catch (const std::exception &erro) {
10     std::cerr << "Erro ao interpretar mensagem MQTT: "
11               << erro.what() << std::endl;
12 }

```

Listing 1: Conversão do payload MQTT em dados internos.

2.3 Threads do núcleo

O arquivo `src/main.cpp` cria os buffers, estados e 11 threads, passa referências com `std::ref` e usa `join()` para controlar o encerramento. As tarefas estão agrupadas abaixo.

Tabela 3: Tarefas concorrentes do núcleo.

Tarefa	Ativação	Função
ComandoNavegacao	80 ms	Atualiza o modo automático/manual do estado global.
ControleNavegacao	80 ms	Executa PID ou comando manual e gera aceleração.
RecebeSensoresMqtt	evento MQTT	Converte sensores e produz duas filas internas.
RecebeComandosMqtt	evento MQTT	Atualiza comando e limite de falha.
PublicaAtuadoresMqtt	80 ms	Envia aceleração e câmera à simulação.
PublicaEstadoMqtt	80 ms	Envia telemetria à operação remota.
DistanciaPercorrida	evento	Converte transições do encoder em posição.
ReconstrucaoSuperficie	evento	Filtra LIDAR, combina posição e detecta falha.
ColetorDados	evento	Calcula confiança e grava CSV.
PublicaSuperficieMqtt	evento	Envia cada ponto à interface remota.
InspecaoCamera	evento de falha	Executa carga computacional por aproximadamente 2 s.

As tarefas da Tabela 3 foram divididas em três grupos. O primeiro contém as tarefas **periódicas**: `ComandoNavegacao`, `ControleNavegacao`, `PublicaAtuadoresMqtt` e `PublicaEstadoMqtt`. Todas usam período de 80 ms. O comando consolida o modo solicitado, o controle calcula a aceleração e os dois publishers enviam, respectivamente, a

saída para a planta e a telemetria para o supervisor. Essa separação mantém o cálculo do PID independente do tempo gasto na comunicação.

O segundo grupo é **orientado à chegada de dados**. `RecebeSensoresMqtt` transforma um payload em duas entradas, uma para o encoder e outra para o LIDAR. Em seguida, `DistanciaPercorrida` converte as transições do encoder em posição, que é combinada com a leitura de distância por `ReconstrucaoSuperficie`. O ponto resultante é enviado tanto a `ColetorDados` quanto a `PublicaSuperficieMqtt`. Em paralelo, `RecebeComandosMqtt` altera os estados usados pelas próximas execuções periódicas. Enquanto não há trabalho, essas threads permanecem bloqueadas em vez de consultar filas continuamente.

O terceiro grupo trata a **inspeção excepcional**. Ao detectar uma variação acima do limite, a reconstrução sinaliza `InspecaoCamera`. O processamento de aproximadamente 2 s ocorre em uma thread própria porque é muito maior que o período de controle. Dessa forma, a câmera consome CPU sem manter mutexes do pipeline e sem bloquear diretamente a produção dos próximos comandos de aceleração.

Com essa divisão, os dados seguem do sensor MQTT para os buffers de sensor e encoder. Depois passam pelo cálculo de posição e pela reconstrução, que produz o CSV, a superfície remota e, quando necessário, o evento de câmera. Na finalização, as flags dos buffers e as notificações acordam as tarefas bloqueadas. O `main` então espera cada thread com `join()` antes de liberar a biblioteca MQTT, evitando que algum consumidor permaneça aguardando um produtor já encerrado.

2.4 Sincronização

O núcleo compartilha filas, comandos, parâmetros, estados e atuadores entre threads. Sem sincronização, duas operações poderiam modificar a mesma estrutura ao mesmo tempo, uma tarefa poderia ler um estado parcialmente atualizado ou um consumidor poderia tentar retirar um item enquanto o produtor altera a fila. A estratégia adotada foi separar os dados por função e associar um mutex próprio a cada estrutura, reduzindo o número de threads que competem pela mesma seção crítica.

Os estados de robô, comando, atuadores, parâmetros e controle do sistema usam `std::lock_guard`. A thread trava o mutex, copia o conjunto de campos para variáveis locais e libera a proteção antes de realizar cálculos ou comunicação. Esse padrão produz uma fotografia coerente do estado e mantém operações mais lentas, como JSON, MQTT, escrita em arquivo e processamento da câmera, fora da região crítica.

Para dados sequenciais, as filas seguem o padrão produtor-consumidor. Cada uma contém `std::queue`, `std::mutex`, `std::condition_variable` e flag de finalização. A fila preserva a ordem das amostras e desacopla pequenas diferenças de velocidade entre produtor e consumidor. O mutex protege apenas a inserção ou retirada. Depois que ele é liberado, a thread executa filtragem, cálculo e I/O sem aumentar a contenção.

```
1 struct SensorBuffer {  
2     std::queue<SensorData> fila_sensor;  
3     std::mutex mutex_sensor;  
4     std::condition_variable dado_disponivel_var;  
5     bool finalizado = false;  
6 };
```

Listing 2: Estrutura de fila sincronizada.

O consumidor usa `std::unique_lock`, necessário porque `wait()` libera o mutex ao bloquear e o `readquire` ao acordar. Enquanto a fila está vazia, a thread não consome

CPU. O predicado trata tanto dado disponível quanto finalização e também protege contra despertares espúrios: acordar não significa automaticamente que há uma amostra válida.

```

1 std::unique_lock<std::mutex> trava(buffer.mutex_sensor);
2 buffer.dado_disponivel_var.wait(trava, [&buffer] {
3     return !buffer.fila_sensor.empty() || buffer.finalizado;
4 });
5
6 if (buffer.fila_sensor.empty() && buffer.finalizado) {
7     break;
8 }
9
10 leitura = buffer.fila_sensor.front();
11 buffer.fila_sensor.pop();

```

Listing 3: Espera bloqueante sem polling.

`std::lock_guard` aplica RAII às seções críticas simples, fazendo com que o desbloqueio ocorra automaticamente ao sair do escopo, inclusive em exceções. Isso reduz erros de desbloqueio, mas não elimina por si só a possibilidade de deadlock. Por esse motivo, o código evita manter dois mutexes simultaneamente. Quando uma tarefa precisa ler comando e estado, cada estrutura é copiada em um escopo separado, sem criar dependência circular entre travas.

`notify_one()` acorda um único consumidor quando chega uma amostra, pois cada fila normal possui um destino definido. Já `notify_all()` é reservado à finalização, quando todas as threads bloqueadas precisam reavaliar seus predicados. Antes da notificação, a flag `finalizado` é alterada sob o mesmo mutex da fila. Essa ordem evita a perda do sinal de término entre a verificação do predicado e o bloqueio. O evento da câmera usa a mesma estratégia, enquanto o callback MQTT utiliza `std::atomic<bool>` para a flag consultada fora dos mutexes dos buffers.

Dois `SurfaceBuffer` independentes realizam o *fan-out* da reconstrução. Um alimenta o CSV e o outro alimenta o publisher MQTT. Se houvesse apenas uma fila com dois consumidores, os pontos seriam divididos entre eles em vez de enviados aos dois destinos. De forma semelhante, o callback de sensores produz `SensorData` e `EncoderData` em filas separadas. Assim, reconstrução e distância avançam de forma independente, sem perder a ordem correspondente das amostras.

Na prática, cada mecanismo cobre uma necessidade específica. O mutex fornece exclusão mútua, a variável de condição sinaliza trabalho sem polling e a flag ou valor atômico torna o encerramento observável. Não foram usados semáforos porque a própria fila armazena a quantidade e o conteúdo do trabalho pendente. Nesse caso, o predicado da variável de condição expressa diretamente quando o consumidor pode prosseguir.

2.5 Periodicidade

O sistema combina tarefas periódicas e tarefas esporádicas. Controle, interpretação de comando, publicação de atuadores e publicação de estado executam a cada 80 ms, equivalentes a 12,5 Hz. Para essas tarefas foi assumido prazo igual ao período: o cálculo e a publicação de um ciclo devem terminar antes da próxima ativação. A física simulada usa o mesmo intervalo, mantendo alinhadas a geração de sensores e a atualização do controle. A interface gráfica pode redesenhar a 60 quadros/s, mas isso não altera o passo físico de 80 ms.

As demais tarefas não precisam acordar periodicamente. Recepção MQTT, distância, reconstrução, coleta, publicação da superfície e câmera são ativadas pela chegada de

mensagem, disponibilidade na fila ou evento de falha. Essa divisão evita reservar ciclos para tarefas sem dados e reduz uso de CPU quando o sistema está ocioso.

As tarefas cíclicas usam `std::chrono::steady_clock` e `sleep_until`. O relógio monotônico não sofre saltos por ajustes do relógio civil. O instante seguinte é incrementado a partir da referência anterior, evitando acumular diretamente o tempo de processamento de cada ciclo, como ocorreria com chamadas sucessivas de `sleep_for`.

```

1 auto proximaExecucao = std::chrono::steady_clock::now();
2 while (sistemaRodando) {
3     proximaExecucao += std::chrono::milliseconds(80);
4     // Leitura, calculo e publicacao.
5     std::this_thread::sleep_until(proximaExecucao);
6 }

```

Listing 4: Padrão usado nas tarefas periódicas.

Em cada iteração, a tarefa calcula primeiro o próximo instante, executa seu trabalho e dorme apenas pelo tempo restante. Se o processamento ultrapassar 80 ms, `sleep_until` retorna imediatamente porque o prazo já passou. Nesse caso, o código inicia o próximo ciclo sem espera para tentar recuperar a referência periódica. Como o protótipo não descarta ciclos nem registra automaticamente cada estouro, a análise temporal posterior mede os custos e verifica a margem disponível em relação ao período.

A câmera é o caso de maior duração, com cerca de 2 s de processamento. Como esse tempo violaria o prazo de uma tarefa periódica, a inspeção foi implementada em uma thread orientada a evento. O escalonador pode então executá-la em paralelo com o controle. Ainda assim, `std::thread` sobre Linux com política normal não oferece garantia de tempo real rígido. O período de 80 ms é uma configuração validada experimentalmente, e não um WCET formal. Essa distinção é retomada na análise de capacidade e escalabilidade.

3 Implementação

3.1 Controle de navegação

No modo automático, `ControleNavegacao` calcula erro, integral limitada e derivada a cada 80 ms. A saturação reduz o *windup*, enquanto a limitação da saída mantém a aceleração no intervalo $[-100, 100]$. Durante a inspeção, o setpoint máximo passa a 1 m/s. No modo manual, os comandos direita, esquerda e parada geram diretamente aceleração positiva, negativa ou nula.

```

1 const double erro = setpointVelocidade - velocidadeAtual;
2 integralErro = std::clamp(
3     integralErro + erro * periodoSegundos,
4     -limiteIntegral,
5     limiteIntegral
6 );
7 const double derivadaErro = (erro - erroAnterior) / periodoSegundos;
8 const double saidaPid = kp * erro + ki * integralErro + kd * derivadaErro;
9 const int aceleracao = static_cast<int>(
10     std::clamp(saidaPid, -100.0, 100.0)
11 );

```

Listing 5: Núcleo do controlador PID.

3.2 Posição e reconstrução

`DistanciaPercorrida` detecta mudanças no nível lógico do encoder. Cada transição representa um metro. O sinal da velocidade define avanço ou retorno, e a posição é limitada a zero. Para cada leitura é produzido um `PositionData`, inclusive quando não ocorre transição, mantendo correspondência um-para-um com as leituras do LIDAR.

`ReconstrucaoSuperficie` retira uma amostra de sensor e uma amostra de posição, aplica média móvel de janela 3 ao LIDAR e cria um ponto (x, y) . A média reduz ruído sem exigir armazenamento completo do percurso.

```

1 ultimasLeituras.push(leitura.i_lidar);
2 somaLeituras += leitura.i_lidar;
3
4 if (ultimasLeituras.size() > tamanhoJanela) {
5     somaLeituras -= ultimasLeituras.front();
6     ultimasLeituras.pop();
7 }
8
9 const double mediaMovel =
10     static_cast<double>(somaLeituras) / ultimasLeituras.size();

```

Listing 6: Média móvel de três leituras.

3.3 Detecção e câmera

A detecção compara leituras brutas consecutivas para não atrasar o evento com o filtro. Se a variação absoluta excede `limite_falha`, o núcleo marca inspeção, liga a câmera e sinaliza `CameraEvent`. A flag `falhaJaDetectada` evita disparos repetidos enquanto a leitura continua fora da faixa.

```

1 const double variacao = std::abs(leitura.i_lidar - leituraAnterior);
2 if (variacao > limiteFalha && !falhaJaDetectada) {
3     falhaJaDetectada = true;
4     robotState.estado.e_inspecao = true;
5     sharedActuatorData.atuadores.o_liga_camera = true;
6     cameraEvent.falha_detectada = true;
7     cameraEvent.camera_event_var.notify_one();
8 }

```

Listing 7: Sinalização de falha.

`InspecaoCamera` fica bloqueada até o evento. Quando acorda, executa funções trigonométricas continuamente por cerca de 2 s. A tarefa representa processamento pesado real, pois ocupa CPU em vez de apenas chamar `sleep`. Ao terminar, desliga a câmera e libera o modo de inspeção. A execução em thread própria impede que esse trabalho bloqueie o controle periódico.

3.4 Coleta de dados

`ColetorDados` grava `timestamp`, x , y e confiança em `surface_points.csv`. A confiança é proporcional à quantidade de pontos anteriores dentro de ± 2 m em x e ± 15 cm em y , saturada em 1 após cinco pontos próximos. O algoritmo é online, porém percorre todo o histórico para cada amostra, característica importante na análise de escalabilidade.

3.5 Inicialização e encerramento

`main.cpp` inicializa a biblioteca Mosquitto, configura os estados iniciais e cria as 11 threads. As referências são passadas com `std::ref`, e os objetos permanecem vivos enquanto

os `join` são executados. O encerramento começa quando o núcleo recebe a mensagem MQTT `finalizado`. A partir dela, os buffers são marcados, os consumidores são acordados e o término é propagado até a reconstrução e a câmera. Somente depois disso as tarefas cíclicas são desativadas.

O script `run_all.py` automatiza compilação, verificação/inicialização do broker, criação dos três processos e encerramento dos grupos de processos ao receber `Ctrl+C`. A execução recomendada é:

```
python3 run_all.py
```

4 Resultados dos Testes

4.1 Ambiente

Os testes apresentados neste relatório foram executados em Ubuntu 24.04, kernel Linux 6.17, processador AMD Ryzen 7 4800H (8 núcleos e 16 processadores lógicos), 14 GiB de RAM, g++ 13.3, Python 3.12.3, Pygame 2.6.1 e Mosquitto 2.0.18. A compilação foi refeita com `-std=c++17 -Wall -Wextra -pthread` e terminou sem erros ou avisos. Todos os arquivos Python passaram por `py_compile`.

4.2 Matriz de testes

Tabela 4: Testes funcionais e resultados.

ID	Situação	Procedimento	Resultado
T1	Compilação e sintaxe	Recompilação forçada do núcleo e <code>py_compile</code> das interfaces.	Aprovado. O binário e os módulos Python foram gerados corretamente.
T2	Fluxo nominal automático	Publicação de sensores normais e verificação de controle, posição, reconstrução, MQTT e CSV.	Aprovado. Os pontos foram consumidos em ordem e o robô acelerou até o setpoint.
T3	Ruído do LIDAR	Leituras normais com ruído gaussiano e média móvel de janela 3.	Aprovado. O perfil apresentou menor variação entre amostras.
T4	Falha severa	Mudança de aproximadamente 100 cm para 128–140 cm, acima do limite.	Aprovado. A inspeção foi ativada, a câmera foi ligada e o setpoint foi limitado a 1 m/s.
T5	Saliência menor	Região de 94 cm com limite configurado para não caracterizar falha severa.	Aprovado no limite nominal. O limiar também pôde ser alterado remotamente.
T6	Operação manual	Seleção manual, direita, esquerda e parada pela interface remota.	Aprovado. Os comandos produziram acelerações 70, -70 e 0.

ID	Situação	Procedimento	Resultado
T7	Finalização	Publicação de {"finalizado":true} e espera por todos os join.	Aprovado. Os consumidores acordaram e o processo encerrou sem espera ocupada.
T8	JSON inválido	Payload incompleto ou malformado no callback MQTT.	A mensagem foi descartada por exceção e o erro foi registrado.
T9	Desempenho	20 rodadas com 2.000 amostras nas tarefas reais isoladas.	Os custos normais ficaram muito abaixo de 80 ms, mesmo com a câmera em uma thread separada.

4.3 Evidências visuais

As Figuras 3 e 4 usam as próprias rotinas de desenho das interfaces com um estado representativo da região de falha. Elas registram quais informações são apresentadas durante a integração: sensores/atuadores na simulação e telemetria/superfície na operação remota.

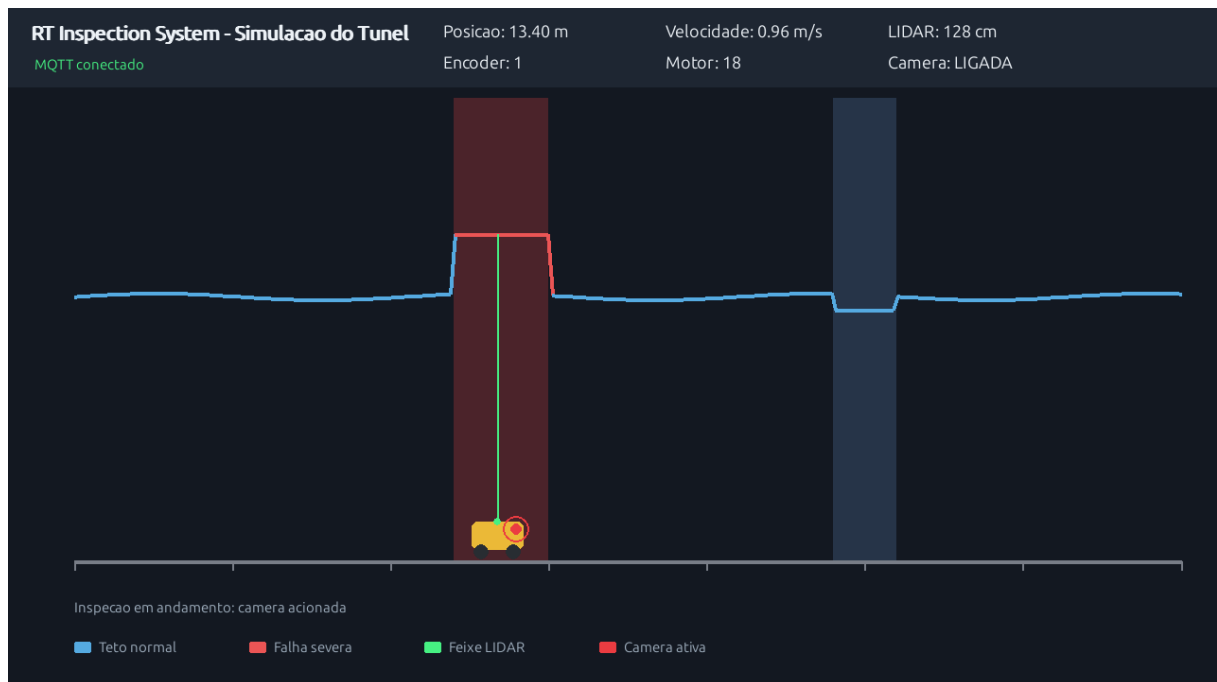


Figura 3: Interface de simulação na região de falha: LIDAR em 128 cm, câmera ligada e velocidade reduzida.

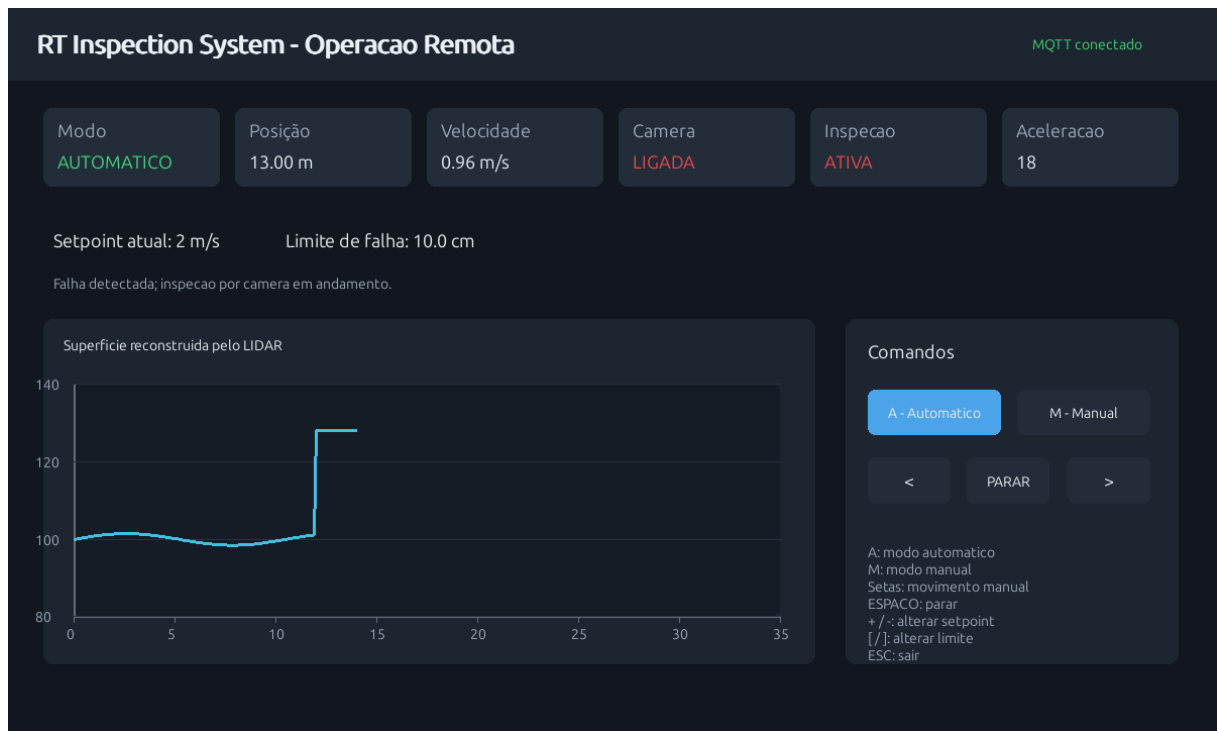


Figura 4: Operação remota exibindo inspeção ativa e descontinuidade na superfície reconstruída.

O teste determinístico publica uma anomalia conhecida e permite verificar o evento sem depender do ruído aleatório. A Figura 5 mostra a falha detectada, o início da câmera e o término do processamento. A Figura 6 mostra o CSV produzido pelo pipeline.

```
leandrolopo@leandrolopo-Nitro-AN515-44:~/ufmg/atr/Trabalho ATR/RT-Inspection-Syst
em$ cat logs/teste_4_4_terminal.txt
$ grep -E "Falha detectada|InspecaoCamera" logs/saida_completa.txt
Falha detectada: x=5 y=115.667 variacao=12.6667 limite=10
InspecaoCamera: iniciando inspecao em x=5 y=115.667 timestamp=2.5006
InspecaoCamera: processamento finalizado resultado=1852.27
leandrolopo@leandrolopo-Nitro-AN515-44:~/ufmg/atr/Trabalho ATR/RT-Inspection-Syst
em$
```

Figura 5: Detecção determinística de falha e execução da tarefa de câmera.

```
$ head -n 8 surface_points.csv
timestamp,x,y,confianca
1.467e-06,0,100,0.2
0.500108,1,100.5,0.4
1.00021,2,101,0.6
1.50033,3,102,0.6
2.00046,4,103,0.6
2.5006,5,115.667,0.6
3.0007,6,116.667,0.6
```

Figura 6: Amostras gravadas em surface_points.csv.

4.4 Limites da validação

Os testes cobrem os caminhos nominais, modos de controle, falha, finalização e parsing inválido. Não foram introduzidas perdas artificiais de pacotes, falhas de máquina ou carga competitiva extrema. Como MQTT usa QoS 0 e as filas são não limitadas, testes futuros devem incluir perda de mensagens, reconexão, saturação de filas e análise com sanitizadores de concorrência.

5 Análise de Capacidade e Escalabilidade

5.1 Método de medição

Foi criado o alvo `make benchmark`, que pré-carrega os buffers e chama as implementações reais de `DistanciaPercorrida`, `ReconstrucaoSuperficie`, `ColetorDados` e `InspecaoCamera`. Cada trecho normal processa 2.000 amostras em 20 rodadas. O tempo é medido com `steady_clock`. Em seguida, a média, o percentil 95 e o maior resultado entre as rodadas são divididos pelo número de amostras. O benchmark também mede o custo de serialização e interpretação do JSON, mantendo os logs redirecionados para que não dominem o resultado.

Essas medidas representam custo computacional com mutexes sem contenção e dados já disponíveis. Elas não incluem atraso do broker, escalonamento entre processos, desenho da interface nem contenção do sistema operacional. Portanto, servem para dimensionamento de CPU, e não como WCET formal.

Tabela 5: Resultados do benchmark temporal.

Trecho	Média (μs)	P95 (μs)	Máximo (μs)
Distância percorrida	1,360	2,183	3,120
Reconstrução da superfície	1,830	1,889	1,915
Coletor de dados	12,453	12,688	12,715
JSON (serializar + interpretar)	21,629	22,231	22,590
Câmera	2.000,256 ms por evento		

Considerando de forma conservadora a soma dos maiores valores normais da Tabela 5, o processamento computacional por amostra é aproximadamente:

$$C_{amostra} \approx 3,120 + 1,915 + 12,715 + 22,590 = 40,340 \mu s \approx 0,041 ms.$$

Com período de 80 ms, a taxa é 12,5 amostras/s e a utilização computacional desse caminho isolado é inferior a 0,1% de um processador lógico. Os publishers de estado e atuadores trabalham no mesmo período. Embora a câmera ocupe aproximadamente um núcleo durante 2 s, ela está separada do caminho de controle. Na máquina com 16 processadores lógicos, essa divisão manteve o controle responsivo durante a inspeção.

5.2 Período mínimo e garantia temporal

O menor período validado para o sistema completo é **80 ms**. Esse valor é usado pela física simulada, pelo controle, pelo comando e pelos publishers, fornecendo margem ampla

sobre o custo computacional medido. Entretanto, essa validação não representa uma garantia de tempo real rígido. O protótipo executa em Linux comum com política padrão `SCHED_OTHER`, sem prioridade de tempo real, afinidade de CPU, memória bloqueada ou medição de WCET sob todas as cargas.

O custo isolado indica que períodos computacionais menores seriam possíveis. Contudo, à medida que o período diminui, atraso do MQTT, desenho, jitter e preempções passam a ter mais influência que o próprio cálculo. Por isso, **80 ms é o menor ciclo testado e considerado seguro nesta configuração**. A adoção de um valor inferior exigiria uma campanha específica de latência e jitter. Uma garantia formal também dependeria de `SCHED_FIFO` ou `SCHED_RR`, prioridades por criticidade, medição de pior caso, filas limitadas e testes sob carga.

5.3 Gargalos de escalabilidade

- **Coletor:** a confiança percorre todo o histórico. Para N pontos, o custo acumulado é $O(N^2)$. Uma janela espacial ou índice por posição tornaria o custo aproximadamente constante por amostra.
- **Filas:** `std::queue` não possui limite nem política de descarte. Se o produtor superar o consumidor por muito tempo, memória e latência crescem. Filas circulares limitadas são mais adequadas para operação contínua.
- **MQTT QoS 0:** prioriza baixa sobrecarga, mas não garante entrega. QoS 1 ou numeração de sequência seria necessária se nenhuma amostra pudesse ser perdida.
- **Logs:** `std::cout` protegido por mutex serializa threads e pode aumentar jitter. Em produção, os logs devem ser reduzidos ou enviados a uma fila assíncrona.
- **Câmera:** a carga de 2 s é proposital e paralela. Em hardware com um único núcleo, ela competiria diretamente com o controle e exigiria prioridade/afinidade ou aceleração dedicada.
- **Broker único:** é adequado ao protótipo local. Vários robôs exigiriam prefixos por dispositivo, controle de acesso e avaliação de vazão do broker.

6 Link para o Vídeo

O vídeo foi produzido a partir de uma execução integrada do sistema, iniciada pelo script `run_all.py`. A gravação reuniu a simulação do túnel e a interface de operação remota, enquanto a narração relacionou o comportamento exibido com o processamento realizado pelo núcleo C++ e com a comunicação MQTT entre os processos.

Com duração inferior a 3 minutos, o vídeo apresenta o movimento do robô em modo automático e a construção progressiva do perfil medido pelo LIDAR. Na região de falha, são mostrados o aumento da distância ao teto, a detecção da variação, o acionamento da câmera e a redução da velocidade durante a inspeção. A demonstração termina com uma alteração para o modo manual, evidenciando que os comandos enviados pela interface remota também são recebidos e aplicados pelo núcleo.

Link do vídeo no YouTube:

INSERIR AQUI O LINK DEFINITIVO DO VIDEO

7 Conclusão

O projeto integrou a simulação do robô, o núcleo de processamento e a operação remota em um único fluxo de inspeção. A simulação envia encoder, LIDAR e velocidade por MQTT. A partir desses dados, o núcleo controla o movimento, calcula a posição, reconstrói o teto e aciona a câmera quando encontra uma variação acima do limite. O estado e os pontos reconstruídos retornam à interface remota e também são registrados em CSV.

Para executar esse fluxo de forma concorrente, o núcleo foi dividido em 11 threads com responsabilidades definidas. Os mutexes protegem os estados compartilhados, enquanto as filas e variáveis de condição coordenam produtores e consumidores sem espera ocupada. As tarefas de controle seguem um período de 80 ms, e o processamento de câmera permanece em uma thread separada para não bloquear diretamente a malha de navegação. Essa organização permitiu relacionar a implementação aos conceitos de concorrência, sincronização, temporização e IPC estudados na disciplina.

Os testes exercitaram o percurso nominal, os modos automático e manual, a reconstrução, a detecção de falha, a câmera e o encerramento das threads. Nas configurações avaliadas, o custo computacional do pipeline normal permaneceu muito abaixo do período adotado, tornando 80 ms o menor ciclo validado para o sistema completo. O resultado confirma o funcionamento do protótipo, mas não caracteriza uma garantia de tempo real rígido. Para aproximar a solução de uma aplicação crítica, ainda seria necessário limitar as filas, melhorar a complexidade do coletor, tratar perdas e reconexões MQTT e executar as tarefas com políticas e prioridades de tempo real.

A Código-fonte

Este apêndice incorpora os arquivos utilizados no projeto. Os códigos são lidos diretamente do repositório durante a compilação deste relatório.

A.1 Compilação

```
CXX := g++

CXXFLAGS := -std=c++17 -Wall -Wextra -pthread -Iinclude
LDLIBS := -lmosquitto

BUILD_DIR := build
TARGET := $(BUILD_DIR)/rt_inspection
BENCHMARK_TARGET := $(BUILD_DIR)/benchmark_temporal

SRCS := $(wildcard src/*.cpp)

.PHONY: all run benchmark clean

all: $(TARGET)

$(TARGET): $(SRCS)
    mkdir -p $(BUILD_DIR)
    $(CXX) $(CXXFLAGS) $(SRCS) -o $(TARGET) $(LDLIBS)

run: $(TARGET)
    ./$(TARGET)

$(BENCHMARK_TARGET): tests/benchmark_temporal.cpp \
    src/distancia_percorrida.cpp src/reconstrucao_superficie.cpp \
    src/coletor_dados.cpp src/inspecao_camera.cpp src/log.cpp
    mkdir -p $(BUILD_DIR)
    $(CXX) $(CXXFLAGS) $^ -o $(BENCHMARK_TARGET)

benchmark: $(BENCHMARK_TARGET)
    cd /tmp && "$(CURDIR)/$(BENCHMARK_TARGET)"

clean:
    rm -rf $(BUILD_DIR) surface_points.csv
```

A.2 Inicialização dos processos

```
1 import shutil
2 import signal
3 import socket
4 import subprocess
5 import sys
6 import time
7 from pathlib import Path
8
9
10 ROOT = Path(__file__).resolve().parent
11 BROKER_HOST = "localhost"
12 BROKER_PORT = 1883
13
14
15 def broker_disponivel() -> bool:
16     try:
17         with socket.create_connection((BROKER_HOST, BROKER_PORT), timeout=1.0):
18             return True
19     except OSError:
20         return False
21
22
23 def iniciar_broker_se_necessario() -> subprocess.Popen | None:
24     if broker_disponivel():
```

```
25     print("Broker MQTT ja esta ativo em localhost:1883.")
26     return None
27
28     mosquitto = shutil.which("mosquitto")
29
30     if mosquitto is None:
31         raise RuntimeError(
32             "Broker MQTT indisponivel. Instale/inicie o Mosquitto "
33             "ou rode: mosquitto -p 1883"
34         )
35
36     print("Iniciando broker MQTT local em localhost:1883...")
37
38     processo = subprocess.Popen(
39         [mosquitto, "-p", str(BROKER_PORT)],
40         cwd=ROOT,
41         start_new_session=True
42     )
43
44     for _ in range(30):
45         if broker_disponivel():
46             return processo
47
48         if processo.poll() is not None:
49             break
50
51         time.sleep(0.1)
52
53     encerrar_processo(processo)
54     raise RuntimeError("Nao foi possivel iniciar o broker MQTT local.")
55
56
57 def iniciar_processo(comando: list[str]) -> subprocess.Popen:
58     print("Executando:", " ".join(comando))
59
60     return subprocess.Popen(
61         comando,
62         cwd=ROOT,
63         start_new_session=True
64     )
65
66
67 def encerrar_processo(processo: subprocess.Popen | None) -> None:
68     if processo is None or processo.poll() is not None:
69         return
70
71     try:
72         os.killpg(processo.pid, signal.SIGTERM)
73         processo.wait(timeout=3.0)
74     except subprocess.TimeoutExpired:
75         os.killpg(processo.pid, signal.SIGKILL)
76         processo.wait()
77
78
79 def os_killpg(pid: int, sig: signal.Signals) -> None:
80     if hasattr(signal, "SIGTERM"):
81         import os
82
83         try:
84             os.killpg(pid, sig)
85         except ProcessLookupError:
86             pass
87
88
89 def solicitar_encerramento(signum, frame) -> None:
90     raise KeyboardInterrupt
91
92
93 def main() -> int:
94     processos: list[subprocess.Popen] = []
95     broker = None
96
97     signal.signal(signal.SIGTERM, solicitar_encerramento)
```

```

98
99     try:
100         subprocess.run(["make"], cwd=ROOT, check=True)
101
102         broker = iniciar_broker_se_necessario()
103
104         processos.append(iniciar_processo(["build/rt_inspection"]))
105         time.sleep(1.0)
106
107         processos.append(
108             iniciar_processo([sys.executable, "simulation/simulation_gui.py"])
109         )
110         processos.append(
111             iniciar_processo([sys.executable, "remote_operation/remote_gui.py"])
112         )
113
114         print("Sistema iniciado. Pressione Ctrl+C para encerrar.")
115
116         while True:
117             for processo in processos:
118                 if processo.poll() is not None:
119                     return processo.returncode or 0
120
121                 time.sleep(0.5)
122
123         except KeyboardInterrupt:
124             print("\nEncerrando processos...")
125             return 0
126
127         finally:
128             for processo in reversed(processos):
129                 encerrar_processo(processo)
130
131             encerrar_processo(broker)
132
133
134 if __name__ == "__main__":
135     raise SystemExit(main())

```

B Cabeçalhos C++

B.1 buffers.hpp

```

1  #pragma once
2
3  #include "types.hpp"
4
5  #include <queue>
6  #include <mutex>
7  #include <condition_variable>
8
9  struct SensorBuffer {
10     std::queue<SensorData> fila_sensor;
11     std::mutex mutex_sensor;
12     std::condition_variable dado_disponivel_var;
13     bool finalizado = false;
14 };
15
16 struct EncoderBuffer {
17     std::queue<EncoderData> fila_encoder;
18     std::mutex mutex_encoder;
19     std::condition_variable encoder_disponivel_var;
20     bool finalizado = false;
21 };
22
23 struct PositionBuffer {
24     std::queue<PositionData> fila_posicao;
25     std::mutex mutex_posicao;
26     std::condition_variable posicao_disponivel_var;

```

```

27     bool finalizado = false;
28 };
29
30 struct SurfaceBuffer {
31     std::queue<SurfacePoint> fila_superficie;
32     std::mutex mutex_superficie;
33     std::condition_variable surface_point_var;
34     bool finalizado = false;
35 };
36
37 struct CameraEvent {
38     std::mutex mutex_camera;
39     std::condition_variable camera_event_var;
40     bool falha_detectada = false;
41     bool finalizado = false;
42     double timestamp = 0.0;
43     double x = 0.0;
44     double y = 0.0;
45 };

```

B.2 log.hpp

```

1 #pragma once
2
3 #include <mutex>
4
5 extern std::mutex coutMutex;

```

B.3 mqtt_topics.hpp

```

1 #pragma once
2
3 namespace MqttTopics {
4     constexpr const char *SENSORES = "atr/sim/sensors";
5     constexpr const char *ATUADORES = "atr/core/actuators";
6     constexpr const char *COMANDOS = "atr/remote/commands";
7     constexpr const char *ESTADO = "atr/core/state";
8     constexpr const char *SUPERFICIE = "atr/core/surface";
9 }

```

B.4 shared_state.hpp

```

1 #pragma once
2
3 #include "types.hpp"
4
5 #include <mutex>
6
7 struct SharedRobotState {
8     RobotState estado;
9     std::mutex mutex_estado;
10 };
11
12 struct SharedCommand {
13     RobotCommand comando;
14     std::mutex mutex_comando;
15 };
16
17 struct SharedActuatorData {
18     ActuatorData atuadores;
19     std::mutex mutex_atuadores;
20 };
21
22 struct SharedSystemParameters {

```

```

23     double limite_falha = 10.0;
24     std::mutex mutex_parametros;
25 };
26
27 struct SharedSystemControl {
28     bool sistema_rodando = true;
29     std::mutex mutex_sistema;
30 };

```

B.5 tasks.hpp

```

1  #pragma once
2
3  #include "buffers.hpp"
4  #include "shared_state.hpp"
5
6  void SimulacaoSensores(SensorBuffer &sensorBuffer,
7                        EncoderBuffer &encoderBuffer);
8
9  void RecebeSensoresMqtt(SensorBuffer &sensorBuffer,
10                         EncoderBuffer &encoderBuffer,
11                         SharedRobotState &robotState);
12
13 void RecebeComandosMqtt(SharedCommand &sharedCommand,
14                        SharedSystemParameters &systemParameters,
15                        SharedSystemControl &systemControl);
16
17 void PublicaAtuadoresMqtt(SharedActuatorData &sharedActuatorData,
18                          SharedSystemControl &systemControl);
19
20 void PublicaEstadoMqtt(SharedRobotState &robotState,
21                      SharedActuatorData &sharedActuatorData,
22                      SharedSystemParameters &systemParameters,
23                      SharedSystemControl &systemControl);
24
25 void PublicaSuperficieMqtt(SurfaceBuffer &remoteSurfaceBuffer);
26
27 void ReconstrucaoSuperficie(SensorBuffer &buffer,
28                             PositionBuffer &positionBuffer,
29                             SurfaceBuffer &surfaceBuffer,
30                             SurfaceBuffer &remoteSurfaceBuffer,
31                             SharedRobotState &robotState,
32                             SharedActuatorData &sharedActuatorData,
33                             SharedSystemParameters &systemParameters,
34                             CameraEvent &cameraEvent);
35
36 void ColetorDados(SurfaceBuffer &surfaceBuffer);
37
38 void DistanciaPercorrida(EncoderBuffer &encoderBuffer,
39                         PositionBuffer &positionBuffer,
40                         SharedRobotState &robotState);
41
42 void ComandoNavegacao(SharedCommand &sharedCommand,
43                      SharedRobotState &robotState,
44                      SharedSystemControl &systemControl);
45
46 void ControleNavegacao(SharedCommand &sharedCommand,
47                       SharedRobotState &robotState,
48                       SharedActuatorData &sharedActuatorData,
49                       SharedSystemControl &systemControl);
50
51 void InspecaoCamera(CameraEvent &cameraEvent,
52                   SharedRobotState &robotState,
53                   SharedActuatorData &sharedActuatorData);

```

B.6 time_utils.hpp

```
1 #pragma once
2
3 double TempoAtualSegundos();
```

B.7 types.hpp

```
1 #pragma once
2
3 struct SensorData {
4     bool i_encoder = false;
5     int i_lidar = 0;
6     double timestamp = 0.0;
7 };
8
9 struct EncoderData {
10     bool i_encoder = false;
11     double timestamp = 0.0;
12 };
13
14 struct PositionData {
15     double timestamp = 0.0;
16     double x = 0.0;
17 };
18
19 struct SurfacePoint {
20     double timestamp = 0.0;
21     double x = 0.0;
22     double y = 0.0;
23     double confianca = 0.0;
24 };
25
26 struct RobotCommand {
27     int j_sp_velocidade = 0;
28     bool c_automatico = false;
29     bool c_man = false;
30     bool c_direita = false;
31     bool c_esquerda = false;
32     bool c_para = false;
33 };
34
35 struct RobotState {
36     bool e_automatico = true;
37     bool e_inspecao = false;
38     double posicao_x = 0.0;
39     double velocidade = 0.0;
40 };
41
42 struct ActuatorData {
43     int o_aceleracao = 0;
44     bool o_liga_camera = false;
45 };
```

C Núcleo C++

C.1 main.cpp

```
1 #include "tasks.hpp"
2
3 #include <mosquitto.h>
4
5 #include <functional>
6 #include <iostream>
7 #include <mutex>
8 #include <thread>
9
```



```

10 int main()
11 {
12     const int resultadoMqtt = mosquitto_lib_init();
13
14     if (resultadoMqtt != MOSQ_ERR_SUCCESS) {
15         std::cerr << "Erro ao inicializar biblioteca MQTT: "
16                 << mosquitto_strerror(resultadoMqtt)
17                 << std::endl;
18
19         return 1;
20     }
21
22     SensorBuffer sensorBuffer;
23     EncoderBuffer encoderBuffer;
24     PositionBuffer positionBuffer;
25
26     SurfaceBuffer surfaceBuffer;
27     SurfaceBuffer remoteSurfaceBuffer;
28
29     CameraEvent cameraEvent;
30
31     SharedRobotState robotState;
32     SharedCommand sharedCommand;
33     SharedActuatorData sharedActuatorData;
34     SharedSystemParameters systemParameters;
35     SharedSystemControl systemControl;
36
37     {
38         std::lock_guard<std::mutex> trava(sharedCommand.mutex_comando);
39
40         sharedCommand.comando.c_automatico = true;
41         sharedCommand.comando.c_man = false;
42         sharedCommand.comando.c_direita = false;
43         sharedCommand.comando.c_esquerda = false;
44         sharedCommand.comando.c_para = false;
45         sharedCommand.comando.j_sp_velocidade = 2;
46     }
47
48     {
49         std::lock_guard<std::mutex> trava(robotState.mutex_estado);
50
51         robotState.estado.e_automatico = true;
52         robotState.estado.e_inspecao = false;
53         robotState.estado.velocidade = 0.0;
54         robotState.estado.posicao_x = 0.0;
55     }
56
57     {
58         std::lock_guard<std::mutex> trava(
59             systemParameters.mutex_parametros
60         );
61
62         systemParameters.limite_falha = 10.0;
63     }
64
65     std::thread comando(
66         ComandoNavegacao,
67         std::ref(sharedCommand),
68         std::ref(robotState),
69         std::ref(systemControl)
70     );
71
72     std::thread controle(
73         ControleNavegacao,
74         std::ref(sharedCommand),
75         std::ref(robotState),
76         std::ref(sharedActuatorData),
77         std::ref(systemControl)
78     );
79
80     std::thread sensoresMqtt(
81         RecebeSensoresMqtt,
82         std::ref(sensorBuffer),

```

```

83         std::ref(encoderBuffer),
84         std::ref(robotState)
85     );
86
87     std::thread comandosMqtt(
88         RecebeComandosMqtt,
89         std::ref(sharedCommand),
90         std::ref(systemParameters),
91         std::ref(systemControl)
92     );
93
94     std::thread atuadoresMqtt(
95         PublicaAtuadoresMqtt,
96         std::ref(sharedActuatorData),
97         std::ref(systemControl)
98     );
99
100    std::thread estadoMqtt(
101        PublicaEstadoMqtt,
102        std::ref(robotState),
103        std::ref(sharedActuatorData),
104        std::ref(systemParameters),
105        std::ref(systemControl)
106    );
107
108    std::thread distancia(
109        DistanciaPercorrida,
110        std::ref(encoderBuffer),
111        std::ref(positionBuffer),
112        std::ref(robotState)
113    );
114
115    std::thread reconstrucao(
116        ReconstrucaoSuperficie,
117        std::ref(sensorBuffer),
118        std::ref(positionBuffer),
119        std::ref(surfaceBuffer),
120        std::ref(remoteSurfaceBuffer),
121        std::ref(robotState),
122        std::ref(sharedActuatorData),
123        std::ref(systemParameters),
124        std::ref(cameraEvent)
125    );
126
127    std::thread coletor(
128        ColetorDados,
129        std::ref(surfaceBuffer)
130    );
131
132    std::thread superficieMqtt(
133        PublicaSuperficieMqtt,
134        std::ref(remoteSurfaceBuffer)
135    );
136
137    std::thread camera(
138        InspecaoCamera,
139        std::ref(cameraEvent),
140        std::ref(robotState),
141        std::ref(sharedActuatorData)
142    );
143
144    sensoresMqtt.join();
145    distancia.join();
146    reconstrucao.join();
147    coletor.join();
148    superficieMqtt.join();
149    camera.join();
150
151    {
152        std::lock_guard<std::mutex> trava(systemControl.mutex_sistema);
153        systemControl.sistema_rodando = false;
154    }
155

```

```

156     comando.join();
157     controle.join();
158
159     comandosMqtt.join();
160     atuadoresMqtt.join();
161     estadoMqtt.join();
162
163     mosquitto_lib_cleanup();
164
165     return 0;
166 }

```

C.2 comando_navegacao.cpp

```

1  #include "log.hpp"
2  #include "shared_state.hpp"
3
4  #include <chrono>
5  #include <iostream>
6  #include <thread>
7
8  void ComandoNavegacao(SharedCommand &sharedCommand,
9                        SharedRobotState &robotState,
10                       SharedSystemControl &systemControl)
11  {
12      auto proximaExecucao = std::chrono::steady_clock::now();
13
14      int ciclo = 0;
15
16      while (true) {
17          proximaExecucao += std::chrono::milliseconds(80);
18
19          {
20              std::lock_guard<std::mutex> trava(systemControl.mutex_sistema);
21
22              if (!systemControl.sistema_rodando) {
23                  break;
24              }
25          }
26
27          RobotCommand comando;
28
29          {
30              std::lock_guard<std::mutex> trava(sharedCommand.mutex_comando);
31              comando = sharedCommand.comando;
32          }
33
34          const bool modoAutomatico =
35              comando.c_automatico && !comando.c_man;
36
37          {
38              std::lock_guard<std::mutex> trava(robotState.mutex_estado);
39              robotState.estado.e_automatico = modoAutomatico;
40          }
41
42          if (ciclo % 10 == 0) {
43              std::lock_guard<std::mutex> trava(coutMutex);
44
45              std::cout << "ComandoNavegacao: automatico="
46                        << modoAutomatico
47                        << " direita=" << comando.c_direita
48                        << " esquerda=" << comando.c_esquerda
49                        << " parar=" << comando.c_para
50                        << " sp_velocidade="
51                        << comando.j_sp_velocidade
52                        << std::endl;
53          }
54
55          ciclo++;
56      }

```

```

57     std::this_thread::sleep_until(proximaExecucao);
58 }
59 }

```

C.3 controle_navegacao.cpp

```

1  #include "log.hpp"
2  #include "shared_state.hpp"
3
4  #include <algorithm>
5  #include <chrono>
6  #include <iostream>
7  #include <thread>
8
9  void ControleNavegacao(SharedCommand &sharedCommand,
10                        SharedRobotState &robotState,
11                        SharedActuatorData &sharedActuatorData,
12                        SharedSystemControl &systemControl)
13  {
14      const double kp = 25.0;
15      const double ki = 1.5;
16      const double kd = 4.0;
17
18      const double periodoSegundos = 0.08;
19      const double limiteIntegral = 20.0;
20
21      double erroAnterior = 0.0;
22      double integralErro = 0.0;
23
24      bool automaticoAnterior = true;
25
26      auto proximaExecucao = std::chrono::steady_clock::now();
27
28      int ciclo = 0;
29
30      while (true) {
31          proximaExecucao += std::chrono::milliseconds(80);
32
33          {
34              std::lock_guard<std::mutex> trava(systemControl.mutex_sistema);
35
36              if (!systemControl.sistema_rodando) {
37                  break;
38              }
39          }
40
41          RobotCommand comando;
42          double velocidadeAtual;
43          bool emInspecao;
44
45          {
46              std::lock_guard<std::mutex> trava(sharedCommand.mutex_comando);
47              comando = sharedCommand.comando;
48          }
49
50          {
51              std::lock_guard<std::mutex> trava(robotState.mutex_estado);
52              velocidadeAtual = robotState.estado.velocidade;
53              emInspecao = robotState.estado.e_inspecao;
54          }
55
56          const bool modoAutomatico =
57              comando.c_automatico && !comando.c_man;
58
59          if (modoAutomatico != automaticoAnterior) {
60              integralErro = 0.0;
61              erroAnterior = 0.0;
62              automaticoAnterior = modoAutomatico;
63          }
64      }

```

```

65     int aceleracao = 0;
66
67     if (modoAutomatico) {
68         const double setpointVelocidade =
69             emInspecao
70             ? std::min(
71                 static_cast<double>(comando.j_sp_velocidade),
72                 1.0
73             )
74             : static_cast<double>(comando.j_sp_velocidade);
75
76         const double erro =
77             setpointVelocidade - velocidadeAtual;
78
79         integralErro = std::clamp(
80             integralErro + erro * periodoSegundos,
81             -limiteIntegral,
82             limiteIntegral
83         );
84
85         const double derivadaErro =
86             (erro - erroAnterior) / periodoSegundos;
87
88         erroAnterior = erro;
89
90         const double saidaPid =
91             kp * erro
92             + ki * integralErro
93             + kd * derivadaErro;
94
95         aceleracao = static_cast<int>(
96             std::clamp(saidaPid, -100.0, 100.0)
97         );
98     }
99     else {
100         integralErro = 0.0;
101         erroAnterior = 0.0;
102
103         if (comando.c_para) {
104             aceleracao = 0;
105             {
106                 std::lock_guard<std::mutex> trava(sharedCommand.mutex_comando);
107                 sharedCommand.comando.j_sp_velocidade = 0;
108             }
109         }
110         else if (comando.c_direita) {
111             aceleracao = 70;
112         }
113         else if (comando.c_esquerda) {
114             aceleracao = -70;
115         }
116         else {
117             aceleracao = 0;
118         }
119     }
120
121     {
122         std::lock_guard<std::mutex> trava(
123             sharedActuatorData.mutex_atuadores
124         );
125
126         sharedActuatorData.atuadores.o_aceleracao = aceleracao;
127         sharedActuatorData.atuadores.o_liga_camera = emInspecao;
128     }
129
130     if (ciclo % 10 == 0) {
131         std::lock_guard<std::mutex> trava(coutMutex);
132
133         std::cout << "ControleNavegacao: modo="
134             << (modoAutomatico ? "automatico" : "manual")
135             << " velocidade=" << velocidadeAtual
136             << " aceleracao=" << aceleracao
137             << " inspecao=" << emInspecao

```

```

138         << std::endl;
139     }
140
141     ciclo++;
142
143     std::this_thread::sleep_until(proximaExecucao);
144 }
145 }

```

C.4 distancia_percorrida.cpp

```

1  #include "buffers.hpp"
2  #include "log.hpp"
3  #include "shared_state.hpp"
4
5  #include <iostream>
6
7  void DistanciaPercorrida(EncoderBuffer &encoderBuffer, PositionBuffer &positionBuffer, SharedRobotState &
   robotState)
8  {
9      bool primeiraLeitura = true;
10     bool encoderAnterior = false;
11
12     while (true)
13     {
14         EncoderData leitura;
15
16         {
17             std::unique_lock<std::mutex> trava(encoderBuffer.mutex_encoder);
18
19             encoderBuffer.encoder_disponivel_var.wait(trava, [&encoderBuffer] {
20                 return !encoderBuffer.fila_encoder.empty() || encoderBuffer.finalizado;
21             });
22
23             if (encoderBuffer.fila_encoder.empty() && encoderBuffer.finalizado)
24             {
25                 break;
26             }
27
28             leitura = encoderBuffer.fila_encoder.front();
29             encoderBuffer.fila_encoder.pop();
30         }
31
32         bool houveMudanca = false;
33
34         if (primeiraLeitura)
35         {
36             encoderAnterior = leitura.i_encoder;
37             primeiraLeitura = false;
38         }
39         else if (leitura.i_encoder != encoderAnterior)
40         {
41             encoderAnterior = leitura.i_encoder;
42             houveMudanca = true;
43         }
44
45         PositionData posicao;
46         posicao.timestamp = leitura.timestamp;
47
48         {
49             std::lock_guard<std::mutex> trava(robotState.mutex_estado);
50
51             if (houveMudanca)
52             {
53                 if (robotState.estado.velocidade >= 0.0)
54                 {
55                     robotState.estado.posicao_x += 1.0;
56                 }
57                 else
58                 {

```

```

59         robotState.estado.posicao_x -= 1.0;
60     }
61
62     if (robotState.estado.posicao_x < 0.0)
63     {
64         robotState.estado.posicao_x = 0.0;
65     }
66 }
67
68 posicao.x = robotState.estado.posicao_x;
69 }
70
71 {
72     std::lock_guard<std::mutex> trava(positionBuffer.mutex_posicao);
73     positionBuffer.fila_posicao.push(posicao);
74 }
75 positionBuffer.posicao_disponivel_var.notify_one();
76
77 if (houveMudanca)
78 {
79     {
80         std::lock_guard<std::mutex> trava(coutMutex);
81         std::cout << "DistanciaPercorrida: x=" << posicao.x
82                   << " m timestamp=" << leitura.timestamp
83                   << std::endl;
84     }
85 }
86 }
87
88 {
89     std::lock_guard<std::mutex> trava(positionBuffer.mutex_posicao);
90     positionBuffer.finalizado = true;
91 }
92 positionBuffer.posicao_disponivel_var.notify_one();
93 }

```

C.5 reconstrucao_superficie.cpp

```

1  #include "buffers.hpp"
2  #include "log.hpp"
3  #include "shared_state.hpp"
4
5  #include <cmath>
6  #include <iostream>
7  #include <queue>
8
9  void ReconstrucaoSuperficie(SensorBuffer &buffer,
10                             PositionBuffer &positionBuffer,
11                             SurfaceBuffer &surfaceBuffer,
12                             SurfaceBuffer &remoteSurfaceBuffer,
13                             SharedRobotState &robotState,
14                             SharedActuatorData &sharedActuatorData,
15                             SharedSystemParameters &systemParameters,
16                             CameraEvent &cameraEvent)
17 {
18     const std::size_t tamanhoJanela = 3;
19
20     std::queue<int> ultimasLeituras;
21
22     int somaLeituras = 0;
23
24     bool primeiraLeitura = true;
25     double leituraAnterior = 0.0;
26     bool falhaJaDetectada = false;
27
28     while (true) {
29         SensorData leitura;
30         PositionData posicao;
31
32         {

```

```

33         std::unique_lock<std::mutex> trava(buffer.mutex_sensor);
34
35         buffer.dado_disponivel_var.wait(
36             trava,
37             [&buffer] {
38                 return !buffer.fila_sensor.empty()
39                     || buffer.finalizado;
40             }
41         );
42
43         if (buffer.fila_sensor.empty() && buffer.finalizado) {
44             break;
45         }
46
47         leitura = buffer.fila_sensor.front();
48         buffer.fila_sensor.pop();
49     }
50
51     {
52         std::unique_lock<std::mutex> trava(
53             positionBuffer.mutex_posicao
54         );
55
56         positionBuffer.posicao_disponivel_var.wait(
57             trava,
58             [&positionBuffer] {
59                 return !positionBuffer.fila_posicao.empty()
60                     || positionBuffer.finalizado;
61             }
62         );
63
64         if (
65             positionBuffer.fila_posicao.empty()
66             && positionBuffer.finalizado
67         ) {
68             break;
69         }
70
71         posicao = positionBuffer.fila_posicao.front();
72         positionBuffer.fila_posicao.pop();
73     }
74
75     ultimasLeituras.push(leitura.i_lidar);
76     somaLeituras += leitura.i_lidar;
77
78     if (ultimasLeituras.size() > tamanhoJanela) {
79         somaLeituras -= ultimasLeituras.front();
80         ultimasLeituras.pop();
81     }
82
83     const double mediaMovel =
84         static_cast<double>(somaLeituras)
85         / static_cast<double>(ultimasLeituras.size());
86
87     SurfacePoint ponto;
88
89     ponto.timestamp = leitura.timestamp;
90     ponto.x = posicao.x;
91     ponto.y = mediaMovel;
92     ponto.confianca = 1.0;
93
94     double limiteFalha;
95
96     {
97         std::lock_guard<std::mutex> trava(
98             systemParameters.mutex_parametros
99         );
100
101         limiteFalha = systemParameters.limite_falha;
102     }
103
104     if (!primeiraLeitura)
105     {

```



```

106     const double variacao =
107         std::abs(leitura.i_lidar - leituraAnterior);
108
109     if (variacao > limiteFalha)
110     {
111         if (!falhaJaDetectada)
112         {
113             falhaJaDetectada = true;
114
115             {
116                 std::lock_guard<std::mutex> trava(
117                     robotState.mutex_estado
118                 );
119
120                 robotState.estado.e_inspecao = true;
121             }
122
123             {
124                 std::lock_guard<std::mutex> trava(
125                     sharedActuatorData.mutex_atuadores
126                 );
127
128                 sharedActuatorData.atuadores.o_liga_camera = true;
129             }
130
131             {
132                 std::lock_guard<std::mutex> trava(
133                     cameraEvent.mutex_camera
134                 );
135
136                 cameraEvent.falha_detectada = true;
137                 cameraEvent.timestamp = ponto.timestamp;
138                 cameraEvent.x = ponto.x;
139                 cameraEvent.y = ponto.y;
140             }
141
142             cameraEvent.camera_event_var.notify_one();
143
144             {
145                 std::lock_guard<std::mutex> trava(coutMutex);
146
147                 std::cout << "Falha detectada: x="
148                     << ponto.x
149                     << " y=" << ponto.y
150                     << " variacao=" << variacao
151                     << " limite=" << limiteFalha
152                     << std::endl;
153             }
154         }
155     }
156     else
157     {
158         falhaJaDetectada = false;
159     }
160 }
161
162 leituraAnterior = leitura.i_lidar;
163 primeiraLeitura = false;
164
165 {
166     std::lock_guard<std::mutex> trava(
167         surfaceBuffer.mutex_superficie
168     );
169
170     surfaceBuffer.fila_superficie.push(ponto);
171 }
172
173 surfaceBuffer.surface_point_var.notify_one();
174
175 {
176     std::lock_guard<std::mutex> trava(
177         remoteSurfaceBuffer.mutex_superficie
178     );

```

```

179         remoteSurfaceBuffer.fila_superficie.push(ponto);
180     }
181
182     remoteSurfaceBuffer.surface_point_var.notify_one();
183
184     {
185         std::lock_guard<std::mutex> trava(coutMutex);
186
187         std::cout << "Reconstrucao: encoder="
188             << leitura.i_encoder
189             << " lidar=" << leitura.i_lidar
190             << " media_movel=" << mediaMovel
191             << " limite_falha=" << limiteFalha
192             << " timestamp=" << leitura.timestamp
193             << std::endl;
194     }
195 }
196
197 {
198     std::lock_guard<std::mutex> trava(surfaceBuffer.mutex_superficie);
199     surfaceBuffer.finalizado = true;
200 }
201
202 surfaceBuffer.surface_point_var.notify_all();
203
204 {
205     std::lock_guard<std::mutex> trava(
206         remoteSurfaceBuffer.mutex_superficie
207     );
208
209     remoteSurfaceBuffer.finalizado = true;
210 }
211
212 remoteSurfaceBuffer.surface_point_var.notify_all();
213
214 {
215     std::lock_guard<std::mutex> trava(cameraEvent.mutex_camera);
216     cameraEvent.finalizado = true;
217 }
218
219 cameraEvent.camera_event_var.notify_all();
220 }
221

```

C.6 coletor_dados.cpp

```

1  #include "buffers.hpp"
2  #include "log.hpp"
3
4  #include <algorithm>
5  #include <cmath>
6  #include <fstream>
7  #include <iostream>
8  #include <vector>
9
10 void ColetorDados(SurfaceBuffer &surfaceBuffer)
11 {
12     std::ofstream arquivo("surface_points.csv");
13     arquivo << "timestamp,x,y,confianca\n";
14     std::vector<SurfacePoint> historico;
15
16     while (true)
17     {
18         SurfacePoint ponto;
19
20         {
21             std::unique_lock<std::mutex> trava(surfaceBuffer.mutex_superficie);
22
23             surfaceBuffer.surface_point_var.wait(trava, [&surfaceBuffer] {
24                 return !surfaceBuffer.fila_superficie.empty() || surfaceBuffer.finalizado;

```

```

25     });
26
27     if (surfaceBuffer.fila_superficie.empty() && surfaceBuffer.finalizado)
28     {
29         break;
30     }
31
32     ponto = surfaceBuffer.fila_superficie.front();
33     surfaceBuffer.fila_superficie.pop();
34 }
35
36 int pontosProximos = 1;
37 for (const SurfacePoint &anterior : historico)
38 {
39     const bool proximoEmX = std::abs(anterior.x - ponto.x) <= 2.0;
40     const bool proximoEmY = std::abs(anterior.y - ponto.y) <= 15.0;
41
42     if (proximoEmX && proximoEmY)
43     {
44         pontosProximos++;
45     }
46 }
47
48 ponto.confianca = std::min(1.0, pontosProximos / 5.0);
49 historico.push_back(ponto);
50
51 {
52     std::lock_guard<std::mutex> trava(coutMutex);
53     std::cout << "Coletor: timestamp=" << ponto.timestamp
54               << " x=" << ponto.x
55               << " y=" << ponto.y
56               << " confianca=" << ponto.confianca
57               << std::endl;
58 }
59
60 arquivo << ponto.timestamp << ","
61         << ponto.x << ","
62         << ponto.y << ","
63         << ponto.confianca << "\n";
64 }
65 }

```

C.7 inspecao_camera.cpp

```

1  #include "buffers.hpp"
2  #include "log.hpp"
3  #include "shared_state.hpp"
4
5  #include <chrono>
6  #include <cmath>
7  #include <cstdint>
8  #include <iostream>
9  #include <thread>
10
11 void InspecaoCamera(CameraEvent &cameraEvent, SharedRobotState &robotState, SharedActuatorData &
    sharedActuatorData)
12 {
13     while (true)
14     {
15         double timestamp = 0.0;
16         double x = 0.0;
17         double y = 0.0;
18
19         {
20             std::unique_lock<std::mutex> trava(cameraEvent.mutex_camera);
21
22             cameraEvent.camera_event_var.wait(trava, [&cameraEvent] {
23                 return cameraEvent.falha_detectada || cameraEvent.finalizado;
24             });
25

```

```

26         if (!cameraEvent.falha_detectada && cameraEvent.finalizado)
27         {
28             break;
29         }
30
31         timestamp = cameraEvent.timestamp;
32         x = cameraEvent.x;
33         y = cameraEvent.y;
34         cameraEvent.falha_detectada = false;
35     }
36
37     {
38         std::lock_guard<std::mutex> trava(coutMutex);
39         std::cout << "InspecaoCamera: iniciando inspecao em x=" << x
40                 << " y=" << y
41                 << " timestamp=" << timestamp
42                 << std::endl;
43     }
44
45     const auto inicioProcessamento = std::chrono::steady_clock::now();
46     const auto fimProcessamento =
47         inicioProcessamento + std::chrono::seconds(2);
48
49     double acumulador = 0.0;
50     std::uint64_t iteracoes = 0;
51
52     while (std::chrono::steady_clock::now() < fimProcessamento)
53     {
54         for (int indice = 0; indice < 10000; indice++)
55         {
56             const double valor =
57                 static_cast<double>(indice) * 0.001 + x + y;
58
59             acumulador += std::sin(valor) * std::cos(valor * 0.5);
60             iteracoes++;
61         }
62     }
63
64     {
65         std::lock_guard<std::mutex> trava(sharedActuatorData.mutex_atuadores);
66         sharedActuatorData.atuadores.o_liga_camera = false;
67     }
68
69     {
70         std::lock_guard<std::mutex> trava(robotState.mutex_estado);
71         robotState.estado.e_inspecao = false;
72     }
73
74     {
75         std::lock_guard<std::mutex> trava(coutMutex);
76         std::cout << "InspecaoCamera: processamento finalizado resultado="
77                 << acumulador
78                 << " iteracoes=" << iteracoes
79                 << std::endl;
80     }
81 }
82 }

```

C.8 mqtt_sensor_subscriber.cpp

```

1  #include "buffers.hpp"
2  #include "log.hpp"
3  #include "mqtt_topics.hpp"
4  #include "shared_state.hpp"
5  #include "types.hpp"
6
7  #include <mosquitto.h>
8  #include <nlohmann/json.hpp>
9
10 #include <atomic>

```

```

11 #include <iostream>
12 #include <string>
13
14 using json = nlohmann::json;
15
16 namespace {
17
18 struct MqttSensorContext {
19     SensorBuffer *sensorBuffer = nullptr;
20     EncoderBuffer *encoderBuffer = nullptr;
21     SharedRobotState *robotState = nullptr;
22     std::atomic<bool> finalizado{false};
23 };
24
25 void FinalizaBuffers(MqttSensorContext &contexto)
26 {
27     {
28         std::lock_guard<std::mutex> trava(contexto.sensorBuffer->mutex_sensor);
29         contexto.sensorBuffer->finalizado = true;
30     }
31     contexto.sensorBuffer->dado_disponivel_var.notify_all();
32
33     {
34         std::lock_guard<std::mutex> trava(contexto.encoderBuffer->mutex_encoder);
35         contexto.encoderBuffer->finalizado = true;
36     }
37     contexto.encoderBuffer->encoder_disponivel_var.notify_all();
38
39     contexto.finalizado = true;
40 }
41
42 void AoReceberMensagem(struct mosquitto *,
43                        void *userdata,
44                        const struct mosquitto_message *message)
45 {
46     auto *contexto = static_cast<MqttSensorContext *>(userdata);
47
48     if (message == nullptr || message->payload == nullptr) {
49         return;
50     }
51
52     const std::string payload(
53         static_cast<char *>(message->payload),
54         static_cast<std::size_t>(message->payloadlen)
55     );
56
57     try {
58         const json mensagem = json::parse(payload);
59
60         if (mensagem.value("finalizado", false)) {
61             {
62                 std::lock_guard<std::mutex> trava(coutMutex);
63                 std::cout << "MQTT: finalizacao dos sensores recebida."
64                     << std::endl;
65             }
66
67             FinalizaBuffers(*contexto);
68             return;
69         }
70
71         SensorData leitura;
72         leitura.i_encoder = mensagem.at("i_encoder").get<bool>();
73         leitura.i_lidar = mensagem.at("i_lidar").get<int>();
74         leitura.timestamp = mensagem.at("timestamp").get<double>();
75
76         EncoderData encoder;
77         encoder.i_encoder = leitura.i_encoder;
78         encoder.timestamp = leitura.timestamp;
79
80         const double velocidadeMedida =
81             mensagem.value("velocidade", 0.0);
82
83     {

```

```

84         std::lock_guard<std::mutex> trava(contexto->robotState->mutex_estado);
85         contexto->robotState->estado.velocidade = velocidadeMedida;
86     }
87
88     {
89         std::lock_guard<std::mutex> trava(contexto->sensorBuffer->mutex_sensor);
90         contexto->sensorBuffer->fila_sensor.push(leitura);
91     }
92     contexto->sensorBuffer->dado_disponivel_var.notify_one();
93
94     {
95         std::lock_guard<std::mutex> trava(contexto->encoderBuffer->mutex_encoder);
96         contexto->encoderBuffer->fila_encoder.push(encoder);
97     }
98     contexto->encoderBuffer->encoder_disponivel_var.notify_one();
99
100    {
101        std::lock_guard<std::mutex> trava(coutMutex);
102        std::cout << "MQTT sensor recebido: encoder="
103                    << leitura.i_encoder
104                    << " lidar=" << leitura.i_lidar
105                    << " velocidade=" << velocidadeMedida
106                    << " timestamp=" << leitura.timestamp
107                    << std::endl;
108    }
109 }
110 catch (const std::exception &erro) {
111     std::lock_guard<std::mutex> trava(coutMutex);
112     std::cerr << "Erro ao interpretar mensagem MQTT: "
113               << erro.what()
114               << " payload=" << payload
115               << std::endl;
116 }
117 }
118 } // namespace
119
120 void RecebeSensoresMqtt(SensorBuffer &sensorBuffer,
121                        EncoderBuffer &encoderBuffer,
122                        SharedRobotState &robotState)
123 {
124     MqttSensorContext contexto;
125     contexto.sensorBuffer = &sensorBuffer;
126     contexto.encoderBuffer = &encoderBuffer;
127     contexto.robotState = &robotState;
128
129     struct mosquitto *cliente =
130         mosquitto_new("rt_inspection_core_sensors", true, &contexto);
131
132     if (cliente == nullptr) {
133         std::cerr << "Erro: nao foi possivel criar cliente MQTT de sensores."
134                   << std::endl;
135         FinalizaBuffers(contexto);
136         return;
137     }
138
139     mosquitto_message_callback_set(cliente, AoReceberMensagem);
140
141     const int resultadoConexao =
142         mosquitto_connect(cliente, "localhost", 1883, 60);
143
144     if (resultadoConexao != MOSQ_ERR_SUCCESS) {
145         std::cerr << "Erro ao conectar receptor de sensores no MQTT: "
146                   << mosquitto_strerror(resultadoConexao)
147                   << std::endl;
148
149         FinalizaBuffers(contexto);
150         mosquitto_destroy(cliente);
151         return;
152     }
153
154     const int resultadoInscricao =
155         mosquitto_subscribe(cliente, nullptr, MqttTopics::SENSORES, 0);

```

```

157
158     if (resultadoInscricao != MOSQ_ERR_SUCCESS) {
159         std::cerr << "Erro ao assinar topico MQTT de sensores: "
160                 << mosquitto_strerror(resultadoInscricao)
161                 << std::endl;
162
163         FinalizaBuffers(contexto);
164         mosquitto_disconnect(cliente);
165         mosquitto_destroy(cliente);
166         return;
167     }
168
169     {
170         std::lock_guard<std::mutex> trava(coutMutex);
171         std::cout << "MQTT conectado. Aguardando sensores no topico: "
172                 << MqttTopics::SENSORES
173                 << std::endl;
174     }
175
176     while (!contexto.finalizado) {
177         const int resultadoLoop = mosquitto_loop(cliente, 100, 1);
178
179         if (resultadoLoop != MOSQ_ERR_SUCCESS) {
180             std::cerr << "Erro no loop MQTT de sensores: "
181                     << mosquitto_strerror(resultadoLoop)
182                     << std::endl;
183
184             FinalizaBuffers(contexto);
185             break;
186         }
187     }
188
189     mosquitto_disconnect(cliente);
190     mosquitto_destroy(cliente);
191 }

```

C.9 mqtt_command_subscriber.cpp

```

1  #include "log.hpp"
2  #include "mqtt_topics.hpp"
3  #include "shared_state.hpp"
4
5  #include <mosquitto.h>
6  #include <nlohmann/json.hpp>
7
8  #include <algorithm>
9  #include <iostream>
10 #include <string>
11
12 using json = nlohmann::json;
13
14 namespace {
15
16 struct MqttCommandContext {
17     SharedCommand *sharedCommand = nullptr;
18     SharedSystemParameters *systemParameters = nullptr;
19 };
20
21 void AoReceberComando(struct mosquitto *,
22                      void *userdata,
23                      const struct mosquitto_message *message)
24 {
25     auto *contexto = static_cast<MqttCommandContext *>(userdata);
26
27     if (message == nullptr || message->payload == nullptr) {
28         return;
29     }
30
31     const std::string payload(
32         static_cast<char *>(message->payload),

```

```

33     static_cast<std::size_t>(message->payloadlen)
34 );
35
36 try {
37     const json mensagem = json::parse(payload);
38
39     RobotCommand comando;
40
41     comando.c_automatico = mensagem.value("c_automatico", true);
42     comando.c_man = mensagem.value("c_man", false);
43     comando.c_direita = mensagem.value("c_direita", false);
44     comando.c_esquerda = mensagem.value("c_esquerda", false);
45     comando.c_para = mensagem.value("c_para", false);
46
47     comando.j_sp_velocidade = std::clamp(
48         mensagem.value("j_sp_velocidade", 2),
49         0,
50         3
51     );
52
53     const double limiteFalha = std::clamp(
54         mensagem.value("limite_falha", 10.0),
55         1.0,
56         50.0
57     );
58
59     {
60         std::lock_guard<std::mutex> trava(
61             contexto->sharedCommand->mutex_comando
62         );
63
64         contexto->sharedCommand->comando = comando;
65     }
66
67     {
68         std::lock_guard<std::mutex> trava(
69             contexto->systemParameters->mutex_parametros
70         );
71
72         contexto->systemParameters->limite_falha = limiteFalha;
73     }
74
75     {
76         std::lock_guard<std::mutex> trava(coutMutex);
77
78         std::cout << "Comando MQTT recebido: automatico="
79             << comando.c_automatico
80             << " manual=" << comando.c_man
81             << " direita=" << comando.c_direita
82             << " esquerda=" << comando.c_esquerda
83             << " parar=" << comando.c_para
84             << " setpoint=" << comando.j_sp_velocidade
85             << " limite_falha=" << limiteFalha
86             << std::endl;
87     }
88 }
89 catch (const std::exception &erro) {
90     std::lock_guard<std::mutex> trava(coutMutex);
91
92     std::cerr << "Erro ao interpretar comando MQTT: "
93         << erro.what()
94         << " payload=" << payload
95         << std::endl;
96 }
97 }
98
99 } // namespace
100
101 void RecebeComandosMqtt(SharedCommand &sharedCommand,
102     SharedSystemParameters &systemParameters,
103     SharedSystemControl &systemControl)
104 {
105     MqttCommandContext contexto;

```



```

106 contexto.sharedCommand = &sharedCommand;
107 contexto.systemParameters = &systemParameters;
108
109 struct mosquitto *cliente =
110     mosquitto_new("rt_inspection_core_commands", true, &contexto);
111
112 if (cliente == nullptr) {
113     std::lock_guard<std::mutex> trava(coutMutex);
114
115     std::cerr << "Erro: nao foi possivel criar cliente MQTT de comandos."
116         << std::endl;
117
118     return;
119 }
120
121 mosquitto_message_callback_set(cliente, AoReceberComando);
122
123 const int resultadoConexao =
124     mosquitto_connect(cliente, "localhost", 1883, 60);
125
126 if (resultadoConexao != MOSQ_ERR_SUCCESS) {
127     std::lock_guard<std::mutex> trava(coutMutex);
128
129     std::cerr << "Erro ao conectar receptor de comandos no MQTT: "
130         << mosquitto_strerror(resultadoConexao)
131         << std::endl;
132
133     mosquitto_destroy(cliente);
134     return;
135 }
136
137 const int resultadoInscricao =
138     mosquitto_subscribe(cliente, nullptr, MqttTopics::COMANDOS, 0);
139
140 if (resultadoInscricao != MOSQ_ERR_SUCCESS) {
141     std::lock_guard<std::mutex> trava(coutMutex);
142
143     std::cerr << "Erro ao assinar topico de comandos: "
144         << mosquitto_strerror(resultadoInscricao)
145         << std::endl;
146
147     mosquitto_disconnect(cliente);
148     mosquitto_destroy(cliente);
149     return;
150 }
151
152 {
153     std::lock_guard<std::mutex> trava(coutMutex);
154
155     std::cout << "Receptor MQTT de comandos conectado. Topico: "
156         << MqttTopics::COMANDOS
157         << std::endl;
158 }
159
160 while (true) {
161     {
162         std::lock_guard<std::mutex> trava(systemControl.mutex_sistema);
163
164         if (!systemControl.sistema_rodando) {
165             break;
166         }
167     }
168
169     const int resultadoLoop = mosquitto_loop(cliente, 100, 1);
170
171     if (resultadoLoop != MOSQ_ERR_SUCCESS) {
172         std::lock_guard<std::mutex> trava(coutMutex);
173
174         std::cerr << "Erro no loop MQTT de comandos: "
175             << mosquitto_strerror(resultadoLoop)
176             << std::endl;
177     }
178 }

```

```

179         break;
180     }
181 }
182
183 mosquitto_disconnect(cliente);
184 mosquitto_destroy(cliente);
185
186 {
187     std::lock_guard<std::mutex> trava(coutMutex);
188
189     std::cout << "Receptor MQTT de comandos finalizado."
190               << std::endl;
191 }
192 }

```

C.10 mqtt_actuator_publisher.cpp

```

1  #include "log.hpp"
2  #include "mqtt_topics.hpp"
3  #include "shared_state.hpp"
4
5  #include <mosquitto.h>
6  #include <nlohmann/json.hpp>
7
8  #include <chrono>
9  #include <iostream>
10 #include <string>
11 #include <thread>
12
13 using json = nlohmann::json;
14
15 void PublicaAtuadoresMqtt(SharedActuatorData &sharedActuatorData,
16                           SharedSystemControl &systemControl)
17 {
18     struct mosquitto *cliente =
19         mosquitto_new("rt_inspection_core_actuators", true, nullptr);
20
21     if (cliente == nullptr) {
22         std::lock_guard<std::mutex> trava(coutMutex);
23         std::cerr << "Erro: nao foi possivel criar cliente MQTT de atuadores."
24                   << std::endl;
25         return;
26     }
27
28     const int resultadoConexao =
29         mosquitto_connect(cliente, "localhost", 1883, 60);
30
31     if (resultadoConexao != MOSQ_ERR_SUCCESS) {
32         std::lock_guard<std::mutex> trava(coutMutex);
33         std::cerr << "Erro ao conectar publicador de atuadores no MQTT: "
34                   << mosquitto_strerror(resultadoConexao)
35                   << std::endl;
36
37         mosquitto_destroy(cliente);
38         return;
39     }
40
41     {
42         std::lock_guard<std::mutex> trava(coutMutex);
43         std::cout << "Publicador MQTT de atuadores conectado. Topico: "
44                 << MqttTopics::ATUADORES
45                 << std::endl;
46     }
47
48     auto proximaExecucao = std::chrono::steady_clock::now();
49
50     while (true) {
51         proximaExecucao += std::chrono::milliseconds(80);
52
53         bool sistemaRodando = true;

```

```

54
55     {
56         std::lock_guard<std::mutex> trava(systemControl.mutex_sistema);
57         sistemaRodando = systemControl.sistema_rodando;
58     }
59
60     if (!sistemaRodando) {
61         break;
62     }
63
64     ActuatorData atuadores;
65
66     {
67         std::lock_guard<std::mutex> trava(sharedActuatorData.mutex_atuadores);
68         atuadores = sharedActuatorData.atuadores;
69     }
70
71     const json mensagem = {
72         {"o_aceleracao", atuadores.o_aceleracao},
73         {"o_liga_camera", atuadores.o_liga_camera}
74     };
75
76     const std::string payload = mensagem.dump();
77
78     const int resultadoPublicacao =
79         mosquitto_publish(
80             cliente,
81             nullptr,
82             MqttTopics::ATUADORES,
83             static_cast<int>(payload.size()),
84             payload.c_str(),
85             0,
86             false
87         );
88
89     if (resultadoPublicacao != MOSQ_ERR_SUCCESS) {
90         std::lock_guard<std::mutex> trava(coutMutex);
91         std::cerr << "Erro ao publicar atuadores: "
92                 << mosquitto_strerror(resultadoPublicacao)
93                 << std::endl;
94     }
95
96     /*
97     * Processa o envio da mensagem para o broker.
98     * Como este cliente apenas publica, um loop curto e suficiente.
99     */
100    const int resultadoLoop = mosquitto_loop(cliente, 10, 1);
101
102    if (resultadoLoop != MOSQ_ERR_SUCCESS) {
103        std::lock_guard<std::mutex> trava(coutMutex);
104        std::cerr << "Erro no loop MQTT de atuadores: "
105                << mosquitto_strerror(resultadoLoop)
106                << std::endl;
107        break;
108    }
109
110    std::this_thread::sleep_until(proximaExecucao);
111 }
112
113 const json mensagemFinal = {
114     {"finalizado", true}
115 };
116
117 const std::string payloadFinal = mensagemFinal.dump();
118
119 mosquitto_publish(
120     cliente,
121     nullptr,
122     MqttTopics::ATUADORES,
123     static_cast<int>(payloadFinal.size()),
124     payloadFinal.c_str(),
125     0,
126     false

```

```

127 );
128
129 mosquito_loop(cliente, 100, 1);
130
131 mosquito_disconnect(cliente);
132 mosquito_destroy(cliente);
133
134 {
135     std::lock_guard<std::mutex> trava(coutMutex);
136     std::cout << "Publicador MQTT de atuadores finalizado."
137                 << std::endl;
138 }
139 }

```

C.11 mqtt_state_publisher.cpp

```

1  #include "log.hpp"
2  #include "mqtt_topics.hpp"
3  #include "shared_state.hpp"
4
5  #include <mosquitto.h>
6  #include <nlohmann/json.hpp>
7
8  #include <chrono>
9  #include <iostream>
10 #include <string>
11 #include <thread>
12
13 using json = nlohmann::json;
14
15 void PublicaEstadoMqtt(SharedRobotState &robotState,
16                       SharedActuatorData &sharedActuatorData,
17                       SharedSystemParameters &systemParameters,
18                       SharedSystemControl &systemControl)
19 {
20     struct mosquitto *cliente =
21         mosquitto_new("rt_inspection_core_state", true, nullptr);
22
23     if (cliente == nullptr) {
24         std::lock_guard<std::mutex> trava(coutMutex);
25
26         std::cerr << "Erro: nao foi possivel criar publicador MQTT de estado."
27                 << std::endl;
28
29         return;
30     }
31
32     const int resultadoConexao =
33         mosquitto_connect(cliente, "localhost", 1883, 60);
34
35     if (resultadoConexao != MOSQ_ERR_SUCCESS) {
36         std::lock_guard<std::mutex> trava(coutMutex);
37
38         std::cerr << "Erro ao conectar publicador de estado no MQTT: "
39                 << mosquitto_strerror(resultadoConexao)
40                 << std::endl;
41
42         mosquitto_destroy(cliente);
43         return;
44     }
45
46     {
47         std::lock_guard<std::mutex> trava(coutMutex);
48
49         std::cout << "Publicador MQTT de estado conectado. Topico: "
50                 << MqttTopics::ESTADO
51                 << std::endl;
52     }
53
54     auto proximaExecucao = std::chrono::steady_clock::now();

```

```

55
56 while (true) {
57     proximaExecucao += std::chrono::milliseconds(80);
58
59     bool sistemaRodando;
60
61     {
62         std::lock_guard<std::mutex> trava(systemControl.mutex_sistema);
63         sistemaRodando = systemControl.sistema_rodando;
64     }
65
66     if (!sistemaRodando) {
67         break;
68     }
69
70     RobotState estado;
71     ActuatorData atuadores;
72     double limiteFalha;
73
74     {
75         std::lock_guard<std::mutex> trava(robotState.mutex_estado);
76         estado = robotState.estado;
77     }
78
79     {
80         std::lock_guard<std::mutex> trava(
81             sharedActuatorData.mutex_atuadores
82         );
83
84         atuadores = sharedActuatorData.atuadores;
85     }
86
87     {
88         std::lock_guard<std::mutex> trava(
89             systemParameters.mutex_parametros
90         );
91
92         limiteFalha = systemParameters.limite_falha;
93     }
94
95     const json mensagem = {
96         {"e_automatico", estado.e_automatico},
97         {"e_inspecao", estado.e_inspecao},
98         {"posicao_x", estado.posicao_x},
99         {"velocidade", estado.velocidade},
100        {"o_aceleracao", atuadores.o_aceleracao},
101        {"o_liga_camera", atuadores.o_liga_camera},
102        {"limite_falha", limiteFalha}
103    };
104
105    const std::string payload = mensagem.dump();
106
107    const int resultadoPublicacao =
108        mosquitto_publish(
109            cliente,
110            nullptr,
111            MqttTopics::ESTADO,
112            static_cast<int>(payload.size()),
113            payload.c_str(),
114            0,
115            false
116        );
117
118    if (resultadoPublicacao != MOSQ_ERR_SUCCESS) {
119        std::lock_guard<std::mutex> trava(coutMutex);
120
121        std::cerr << "Erro ao publicar estado: "
122            << mosquitto_strerror(resultadoPublicacao)
123            << std::endl;
124    }
125
126    mosquitto_loop(cliente, 10, 1);
127

```

```

128     std::this_thread::sleep_until(proximaExecucao);
129 }
130
131 const json mensagemFinal = {
132     {"finalizado", true}
133 };
134
135 const std::string payloadFinal = mensagemFinal.dump();
136
137 mosquitto_publish(
138     cliente,
139     nullptr,
140     MqttTopics::ESTADO,
141     static_cast<int>(payloadFinal.size()),
142     payloadFinal.c_str(),
143     0,
144     false
145 );
146
147 mosquitto_loop(cliente, 100, 1);
148
149 mosquitto_disconnect(cliente);
150 mosquitto_destroy(cliente);
151
152 {
153     std::lock_guard<std::mutex> trava(coutMutex);
154
155     std::cout << "Publicador MQTT de estado finalizado."
156               << std::endl;
157 }
158 }

```

C.12 mqtt_surface_publisher.cpp

```

1  #include "buffers.hpp"
2  #include "log.hpp"
3  #include "mqtt_topics.hpp"
4
5  #include <mosquitto.h>
6  #include <nlohmann/json.hpp>
7
8  #include <iostream>
9  #include <string>
10
11 using json = nlohmann::json;
12
13 void PublicaSuperficieMqtt(SurfaceBuffer &remoteSurfaceBuffer)
14 {
15     struct mosquitto *cliente =
16         mosquitto_new("rt_inspection_core_surface", true, nullptr);
17
18     if (cliente == nullptr) {
19         std::lock_guard<std::mutex> trava(coutMutex);
20
21         std::cerr << "Erro: nao foi possivel criar publicador MQTT de superficie."
22                   << std::endl;
23
24         return;
25     }
26
27     const int resultadoConexao =
28         mosquitto_connect(cliente, "localhost", 1883, 60);
29
30     if (resultadoConexao != MOSQ_ERR_SUCCESS) {
31         std::lock_guard<std::mutex> trava(coutMutex);
32
33         std::cerr << "Erro ao conectar publicador de superficie no MQTT: "
34                   << mosquitto_strerror(resultadoConexao)
35                   << std::endl;
36

```

```

37     mosquito_destroy(cliente);
38     return;
39 }
40
41 {
42     std::lock_guard<std::mutex> trava(coutMutex);
43
44     std::cout << "Publicador MQTT de superficie conectado. Topico: "
45               << MqttTopics::SUPERFICIE
46               << std::endl;
47 }
48
49 while (true) {
50     SurfacePoint ponto;
51
52     {
53         std::unique_lock<std::mutex> trava(
54             remoteSurfaceBuffer.mutex_superficie
55         );
56
57         remoteSurfaceBuffer.surface_point_var.wait(
58             trava,
59             [&remoteSurfaceBuffer] {
60                 return !remoteSurfaceBuffer.fila_superficie.empty()
61                     || remoteSurfaceBuffer.finalizado;
62             }
63         );
64
65         if (
66             remoteSurfaceBuffer.fila_superficie.empty()
67             && remoteSurfaceBuffer.finalizado
68         ) {
69             break;
70         }
71
72         ponto = remoteSurfaceBuffer.fila_superficie.front();
73         remoteSurfaceBuffer.fila_superficie.pop();
74     }
75
76     const json mensagem = {
77         {"timestamp", ponto.timestamp},
78         {"x", ponto.x},
79         {"y", ponto.y},
80         {"confianca", ponto.confianca}
81     };
82
83     const std::string payload = mensagem.dump();
84
85     const int resultadoPublicacao =
86         mosquito_publish(
87             cliente,
88             nullptr,
89             MqttTopics::SUPERFICIE,
90             static_cast<int>(payload.size()),
91             payload.c_str(),
92             0,
93             false
94         );
95
96     if (resultadoPublicacao != MOSQ_ERR_SUCCESS) {
97         std::lock_guard<std::mutex> trava(coutMutex);
98
99         std::cerr << "Erro ao publicar ponto da superficie: "
100                << mosquito_strerror(resultadoPublicacao)
101                << std::endl;
102     }
103
104     mosquito_loop(cliente, 10, 1);
105 }
106
107 const json mensagemFinal = {
108     {"finalizado", true}
109 };

```

```

110
111     const std::string payloadFinal = mensagemFinal.dump();
112
113     mosquitto_publish(
114         cliente,
115         nullptr,
116         MqttTopics::SUPERFICIE,
117         static_cast<int>(payloadFinal.size()),
118         payloadFinal.c_str(),
119         0,
120         false
121     );
122
123     mosquitto_loop(cliente, 100, 1);
124
125     mosquitto_disconnect(cliente);
126     mosquitto_destroy(cliente);
127
128     {
129         std::lock_guard<std::mutex> trava(coutMutex);
130
131         std::cout << "Publicador MQTT de superficie finalizado."
132                 << std::endl;
133     }
134 }

```

C.13 simulacao_sensores.cpp

```

1  #include "buffers.hpp"
2  #include "time_utils.hpp"
3  #include "types.hpp"
4
5  #include <chrono>
6  #include <thread>
7
8  void SimulacaoSensores(SensorBuffer &sensorBuffer, EncoderBuffer &encoderBuffer)
9  {
10     bool estado_encoder = false;
11     for (int i = 0; i < 10; i++)
12     {
13         estado_encoder = !estado_encoder;
14
15         SensorData leitura;
16         leitura.i_encoder = estado_encoder;
17         leitura.i_lidar = (i == 5) ? 140 : 100 + i;
18         leitura.timestamp = TempoAtualSegundos();
19
20         EncoderData encoder;
21         encoder.i_encoder = leitura.i_encoder;
22         encoder.timestamp = leitura.timestamp;
23
24         {
25             std::lock_guard<std::mutex> trava(encoderBuffer.mutex_encoder);
26             encoderBuffer.fila_encoder.push(encoder);
27         }
28
29         encoderBuffer.encoder_disponivel_var.notify_one();
30
31         {
32             std::lock_guard<std::mutex> trava(sensorBuffer.mutex_sensor);
33             sensorBuffer.fila_sensor.push(leitura);
34         }
35
36         sensorBuffer.dado_disponivel_var.notify_one();
37         std::this_thread::sleep_for(std::chrono::milliseconds(500));
38     }
39
40     {
41         std::lock_guard<std::mutex> trava(sensorBuffer.mutex_sensor);
42         sensorBuffer.finalizado = true;

```



```

43     }
44
45     sensorBuffer.dado_disponivel_var.notify_one();
46
47     {
48         std::lock_guard<std::mutex> trava(encoderBuffer.mutex_encoder);
49         encoderBuffer.finalizado = true;
50     }
51
52     encoderBuffer.encoder_disponivel_var.notify_one();
53 }

```

C.14 log.cpp

```

1 #include "log.hpp"
2
3 std::mutex coutMutex;

```

C.15 time_utils.cpp

```

1 #include "time_utils.hpp"
2
3 #include <chrono>
4
5 double TempoAtualSegundos()
6 {
7     using Clock = std::chrono::steady_clock;
8
9     static const auto inicio = Clock::now();
10    const auto agora = Clock::now();
11    const auto tempoDecorrido = agora - inicio;
12
13    return std::chrono::duration<double>(tempoDecorrido).count();
14 }

```

D Simulação e Operação Remota

D.1 simulation_gui.py

```

1 import json
2 import math
3 import random
4 import time
5 from dataclasses import dataclass
6 from threading import Lock
7
8 import paho.mqtt.client as mqtt
9 import pygame
10
11
12 # =====
13 # Configuracoes MQTT
14 # =====
15
16 BROKER = "localhost"
17 PORTA = 1883
18
19 TOPICO_SENSORES = "atr/sim/sensors"
20 TOPICO_ATUADORES = "atr/core/actuators"
21
22
23 # =====

```

```

24 # Configuracoes da simulacao fisica
25 # =====
26
27 PERIODO_SIMULACAO = 0.08      # 80 ms
28 COMPRIMENTO_TUNEL = 35.0     # metros
29
30 VELOCIDADE_MAXIMA = 3.0       # m/s
31 ACELERACAO_MAXIMA = 1.8       # m/s2
32 ATRITO = 0.35                 # resistencia proporcional a velocidade
33
34 DISTANCIA_NORMAL_TETO = 100.0 # centimetros
35 DESVIO_PADRAO_RUIDO = 0.8    # centimetros
36
37
38 # =====
39 # Configuracoes graficas
40 # =====
41
42 LARGURA_JANELA = 1280
43 ALTURA_JANELA = 720
44
45 MARGEM_ESQUERDA = 70
46 MARGEM_DIREITA = 40
47
48 Y_CHAO = 585
49 Y_SENSOR_ROBO = 540
50
51 LARGURA_AREA_TUNEL = (
52     LARGURA_JANELA - MARGEM_ESQUERDA - MARGEM_DIREITA
53 )
54
55 ESCALA_VERTICAL_LIDAR = 2.35
56
57 COR_FUNDO = (19, 24, 33)
58 COR_PAINEL = (30, 38, 50)
59 COR_TEXTO = (235, 240, 245)
60 COR_TEXTO_SECUNDARIO = (160, 175, 190)
61
62 COR_CHAO = (120, 125, 135)
63 COR_TETO_NORMAL = (85, 170, 225)
64 COR_TETO_FALHA = (235, 85, 85)
65 COR_LIDAR = (70, 240, 130)
66
67 COR_ROBO = (235, 185, 55)
68 COR_RODA = (40, 45, 50)
69 COR_CAMERA_DESLIGADA = (90, 100, 110)
70 COR_CAMERA_LIGADA = (235, 60, 65)
71
72 COR_AREA_FALHA = (75, 35, 42)
73 COR_AREA_SALIENCIA = (38, 52, 72)
74
75
76 # =====
77 # Atuadores recebidos do nucleo C++
78 # =====
79
80 @dataclass
81 class Atuadores:
82     aceleracao: int = 0
83     camera_ligada: bool = False
84
85
86 # =====
87 # Simulacao do robo e do tunel
88 # =====
89
90 class SimulacaoTunel:
91     def __init__(self) -> None:
92         self.x = 0.0
93         self.velocidade = 0.0
94
95         self.estado_encoder = False
96         self.ultimo_metro_encoder = 0

```

```

97
98     self.atuadores = Atuadores()
99     self.mutex_atuadores = Lock()
100
101     self.inicio = time.monotonic()
102
103     self.sensor_atual = {
104         "i_encoder": False,
105         "i_lidar": 100,
106         "velocidade": 0.0,
107         "posicao_real": 0.0,
108         "timestamp": 0.0
109     }
110
111     self.mqtt_conectado = False
112     self.finalizado = False
113     self.mensagem_status = "Aguardando conexao MQTT..."
114
115     def distancia_teto(self, x: float) -> float:
116         """
117         Distancia entre o sensor do robo e o teto, em centimetros.
118
119         12 m a 15 m:
120             Falha severa: cavidade no teto.
121
122         24 m a 26 m:
123             Pequena saliencia, menor que a falha principal.
124         """
125         if 12.0 <= x <= 15.0:
126             return 128.0
127
128         if 24.0 <= x <= 26.0:
129             return 94.0
130
131         ondulacao = 1.5 * math.sin(x * 0.6)
132         return DISTANCIA_NORMAL_TETO + ondulacao
133
134     def atualizar_encoder(self) -> None:
135         metro_atual = int(self.x)
136
137         while self.ultimo_metro_encoder < metro_atual:
138             self.estado_encoder = not self.estado_encoder
139             self.ultimo_metro_encoder += 1
140
141         while self.ultimo_metro_encoder > metro_atual:
142             self.estado_encoder = not self.estado_encoder
143             self.ultimo_metro_encoder -= 1
144
145     def atualizar_movimento(self, dt: float) -> None:
146         with self.mutex_atuadores:
147             comando_motor = self.atuadores.aceleracao
148
149             aceleracao_motor = (
150                 comando_motor / 100.0
151             ) * ACELERACAO_MAXIMA
152
153             aceleracao_resultante = (
154                 aceleracao_motor - ATRITO * self.velocidade
155             )
156
157             self.velocidade += aceleracao_resultante * dt
158
159             self.velocidade = max(
160                 -VELOCIDADE_MAXIMA,
161                 min(self.velocidade, VELOCIDADE_MAXIMA)
162             )
163
164             self.x += self.velocidade * dt
165
166             if self.x <= 0.0:
167                 self.x = 0.0
168                 self.velocidade = max(0.0, self.velocidade)
169

```

```

170         if self.x >= COMPRIMENTO_TUNEL:
171             self.x = COMPRIMENTO_TUNEL
172             self.finalizado = True
173
174         self.atualizar_encoder()
175
176     def gerar_sensor(self) -> dict:
177         distancia_real = self.distancia_teto(self.x)
178         ruído = random.gauss(0.0, DESVIO_PADRAO_RUIDO)
179
180         leitura_lidar = int(round(distancia_real + ruído))
181
182         timestamp = round(time.monotonic() - self.inicio, 3)
183
184         self.sensor_atual = {
185             "i_encoder": self.estado_encoder,
186             "i_lidar": leitura_lidar,
187             "velocidade": round(self.velocidade, 3),
188             "posicao_real": round(self.x, 3),
189             "timestamp": timestamp
190         }
191
192         return self.sensor_atual
193
194     def receber_atuadores(self, mensagem: dict) -> None:
195         if mensagem.get("finalizado", False):
196             return
197
198         aceleracao = int(mensagem.get("o_aceleracao", 0))
199         camera_ligada = bool(mensagem.get("o_liga_camera", False))
200
201         with self.mutex_atuadores:
202             self.atuadores.aceleracao = aceleracao
203             self.atuadores.camera_ligada = camera_ligada
204
205         if camera_ligada:
206             self.mensagem_status = "Inspecao em andamento: camera acionada"
207         else:
208             self.mensagem_status = "Simulacao em execucao"
209
210
211     # =====
212     # Conversoes de coordenadas
213     # =====
214
215     def x_para_tela(x: float) -> int:
216         proporcao = x / COMPRIMENTO_TUNEL
217
218         return int(
219             MARGEM_ESQUERDA +
220             proporcao * LARGURA_AREA_TUNEL
221         )
222
223
224     def distancia_para_y_teto(distancia: float) -> int:
225         return int(
226             Y_SENSOR_ROBO -
227             distancia * ESCALA_VERTICAL_LIDAR
228         )
229
230
231     # =====
232     # Desenho da interface
233     # =====
234
235     def desenhar_texto(
236         tela: pygame.Surface,
237         fonte: pygame.font.Font,
238         texto: str,
239         posicao: tuple[int, int],
240         cor: tuple[int, int, int] = COR_TEXTO
241     ) -> None:
242         superficie = fonte.render(texto, True, cor)

```

```

243     tela.blit(superficie, posicao)
244
245
246 def desenhar_areas_especiais(tela: pygame.Surface) -> None:
247     inicio_falha = x_para_tela(12.0)
248     fim_falha = x_para_tela(15.0)
249
250     pygame.draw.rect(
251         tela,
252         COR_AREA_FALHA,
253         pygame.Rect(
254             inicio_falha,
255             95,
256             fim_falha - inicio_falha,
257             Y_CHAO - 95
258         )
259     )
260
261     inicio_salien = x_para_tela(24.0)
262     fim_salien = x_para_tela(26.0)
263
264     pygame.draw.rect(
265         tela,
266         COR_AREA_SALIENCIA,
267         pygame.Rect(
268             inicio_salien,
269             95,
270             fim_salien - inicio_salien,
271             Y_CHAO - 95
272         )
273     )
274
275
276 def desenhar_tunel(
277     tela: pygame.Surface,
278     simulacao: SimulacaoTunel
279 ) -> None:
280     pontos_teto = []
281
282     quantidade_amostras = 250
283
284     for indice in range(quantidade_amostras + 1):
285         x_real = (
286             COMPRIMENTO_TUNEL *
287             indice /
288             quantidade_amostras
289         )
290
291         distancia = simulacao.distancia_teto(x_real)
292
293         pontos_teto.append(
294             (
295                 x_para_tela(x_real),
296                 distancia_para_y_teto(distancia)
297             )
298         )
299
300     for indice in range(len(pontos_teto) - 1):
301         x_real = (
302             COMPRIMENTO_TUNEL *
303             indice /
304             quantidade_amostras
305         )
306
307         cor = (
308             COR_TETO_FALHA
309             if 12.0 <= x_real <= 15.0
310             else COR_TETO_NORMAL
311         )
312
313         pygame.draw.line(
314             tela,
315             cor,

```

```

316         pontos_teto[indice],
317         pontos_teto[indice + 1],
318         4
319     )
320
321     pygame.draw.line(
322         tela,
323         COR_CHAO,
324         (MARGEM_ESQUERDA, Y_CHAO),
325         (LARGURA_JANELA - MARGEM_DIREITA, Y_CHAO),
326         4
327     )
328
329     for metro in range(0, int(Comprimento_Tunnel) + 1, 5):
330         x_tela = x_para_tela(float(metro))
331
332         pygame.draw.line(
333             tela,
334             COR_CHAO,
335             (x_tela, Y_CHAO),
336             (x_tela, Y_CHAO + 9),
337             2
338         )
339
340     def desenhar_robo(
341         tela: pygame.Surface,
342         simulacao: SimulacaoTunel
343     ) -> None:
344         x_robo = x_para_tela(simulacao.x)
345
346         largura_robo = 55
347         altura_robo = 30
348
349         corpo = pygame.Rect(
350             x_robo - largura_robo // 2,
351             Y_CHAO - altura_robo - 12,
352             largura_robo,
353             altura_robo
354         )
355
356         pygame.draw.rect(
357             tela,
358             COR_ROBO,
359             corpo,
360             border_radius=6
361         )
362
363         pygame.draw.circle(
364             tela,
365             COR_RODA,
366             (x_robo - 17, Y_CHAO - 10),
367             8
368         )
369
370         pygame.draw.circle(
371             tela,
372             COR_RODA,
373             (x_robo + 17, Y_CHAO - 10),
374             8
375         )
376
377         sensor_x = x_robo
378         sensor_y = corpo.top
379
380         distancia_sensor = simulacao.sensor_atual["i_lidar"]
381         y_teto_medido = distancia_para_y_teto(distancia_sensor)
382
383         pygame.draw.circle(
384             tela,
385             COR_LIDAR,
386             (sensor_x, sensor_y),
387             4
388         )

```

```

389 )
390
391 pygame.draw.line(
392     tela,
393     COR_LIDAR,
394     (sensor_x, sensor_y),
395     (sensor_x, y_teto_medido),
396     2
397 )
398
399 with simulacao.mutex_atuadores:
400     camera_ligada = simulacao.atuadores.camera_ligada
401
402 cor_camera = (
403     COR_CAMERA_LIGADA
404     if camera_ligada
405     else COR_CAMERA_DESLIGADA
406 )
407
408 pygame.draw.circle(
409     tela,
410     cor_camera,
411     (corpo.right - 8, corpo.top + 8),
412     6
413 )
414
415 if camera_ligada:
416     pygame.draw.circle(
417         tela,
418         COR_CAMERA_LIGADA,
419         (corpo.right - 8, corpo.top + 8),
420         14,
421         2
422     )
423
424
425 def desenhar_painel(
426     tela: pygame.Surface,
427     fonte_titulo: pygame.font.Font,
428     fonte: pygame.font.Font,
429     fonte_pequena: pygame.font.Font,
430     simulacao: SimulacaoTunel
431 ) -> None:
432     pygame.draw.rect(
433         tela,
434         COR_PAINEL,
435         pygame.Rect(0, 0, LARGURA_JANELA, 84)
436     )
437
438     desenhar_texto(
439         tela,
440         fonte_titulo,
441         "RT Inspection System - Simulacao do Tunel",
442         (24, 14)
443     )
444
445     status_cor = (
446         COR_LIDAR
447         if simulacao.mqtt_conectado
448         else COR_CAMERA_LIGADA
449     )
450
451     status_mqtt = (
452         "MQTT conectado"
453         if simulacao.mqtt_conectado
454         else "MQTT desconectado"
455     )
456
457     desenhar_texto(
458         tela,
459         fonte_pequena,
460         status_mqtt,
461         (28, 51),

```

```

462         status_cor
463     )
464
465     sensor = simulacao.sensor_atual
466
467     with simulacao.mutex_atuadores:
468         aceleracao = simulacao.atuadores.aceleracao
469         camera_ligada = simulacao.atuadores.camera_ligada
470
471     valores = [
472         f"Posicao: {sensor['posicao_real']:.2f} m",
473         f"Velocidade: {sensor['velocidade']:.2f} m/s",
474         f"LIDAR: {sensor['i_lidar']} cm",
475         f"Encoder: {int(sensor['i_encoder'])}",
476         f"Motor: {aceleracao}",
477         f"Camera: {'LIGADA' if camera_ligada else 'DESLIGADA'}"
478     ]
479
480     x_inicial = 460
481
482     for indice, valor in enumerate(valores):
483         desenhar_texto(
484             tela,
485             fonte,
486             valor,
487             (x_inicial + (indice % 3) * 235, 14 + (indice // 3) * 33)
488         )
489
490         desenhar_texto(
491             tela,
492             fonte_pequena,
493             simulacao.mensagem_status,
494             (MARGEM_ESQUERDA, 625),
495             COR_TEXTO_SECUNDARIO
496         )
497
498
499     def desenhar_legenda(
500         tela: pygame.Surface,
501         fonte_pequena: pygame.font.Font
502     ) -> None:
503         itens = [
504             ("Teto normal", COR_TETO_NORMAL),
505             ("Falha severa", COR_TETO_FALHA),
506             ("Feixe LIDAR", COR_LIDAR),
507             ("Camera ativa", COR_CAMERA_LIGADA)
508         ]
509
510         x_atual = MARGEM_ESQUERDA
511
512         for texto, cor in itens:
513             pygame.draw.rect(
514                 tela,
515                 cor,
516                 pygame.Rect(x_atual, 670, 18, 12),
517                 border_radius=2
518             )
519
520             desenhar_texto(
521                 tela,
522                 fonte_pequena,
523                 texto,
524                 (x_atual + 26, 667),
525                 COR_TEXTO_SECUNDARIO
526             )
527
528             x_atual += 185
529
530
531     def desenhar_tela_final(
532         tela: pygame.Surface,
533         fonte_titulo: pygame.font.Font,
534         fonte: pygame.font.Font,

```



```

535     simulacao: SimulacaoTunel
536 ) -> None:
537     superficie = pygame.Surface(
538         (LARGURA_JANELA, ALTURA_JANELA),
539         pygame.SRCALPHA
540     )
541
542     superficie.fill((0, 0, 0, 175))
543     tela.blit(superficie, (0, 0))
544
545     texto_final = "Simulacao finalizada"
546
547     largura_texto = fonte_titulo.size(texto_final)[0]
548
549     desenhar_texto(
550         tela,
551         fonte_titulo,
552         texto_final,
553         (
554             LARGURA_JANELA // 2 - largura_texto // 2,
555             ALTURA_JANELA // 2 - 45
556         )
557     )
558
559     resultado = (
560         f"Tunel percorrido: {simulacao.x:.2f} m | "
561         f"Velocidade final: {simulacao.velocidade:.2f} m/s"
562     )
563
564     largura_resultado = fonte.size(resultado)[0]
565
566     desenhar_texto(
567         tela,
568         fonte,
569         resultado,
570         (
571             LARGURA_JANELA // 2 - largura_resultado // 2,
572             ALTURA_JANELA // 2 + 8
573         ),
574         COR_TEXTO_SECUNDARIO
575     )
576
577
578 # =====
579 # MQTT
580 # =====
581
582 simulacao = SimulacaoTunel()
583
584
585 def ao_conectar(
586     cliente: mqtt.Client,
587     dados_usuario,
588     flags,
589     codigo_motivo,
590     propriedades
591 ) -> None:
592     simulacao.mqtt_conectado = True
593     simulacao.mensagem_status = "MQTT conectado. Aguardando atuadores do nucleo C++."
594
595     cliente.subscribe(TOPICO_ATUADORES)
596
597     print(f"Simulacao conectada ao broker MQTT: {codigo_motivo}")
598     print(f"Assinando atuadores em: {TOPICO_ATUADORES}")
599     print(f"Publicando sensores em: {TOPICO_SENSORES}")
600
601
602 def ao_desconectar(
603     cliente: mqtt.Client,
604     dados_usuario,
605     flags_desconexao,
606     codigo_motivo,
607     propriedades

```

```

608 ) -> None:
609     simulacao.mqtt_conectado = False
610     simulacao.mensagem_status = "Conexao MQTT encerrada."
611
612
613 def ao_receber_mensagem(
614     cliente: mqtt.Client,
615     dados_usuario,
616     mensagem: mqtt.MQTTMessage
617 ) -> None:
618     try:
619         payload = mensagem.payload.decode("utf-8")
620         dados = json.loads(payload)
621
622         simulacao.receber_atuadores(dados)
623
624     except (UnicodeDecodeError, json.JSONDecodeError, ValueError) as erro:
625         simulacao.mensagem_status = f"Erro ao receber atuadores: {erro}"
626
627
628 # =====
629 # Execucao principal
630 # =====
631
632 def main() -> None:
633     pygame.init()
634
635     tela = pygame.display.set_mode(
636         (LARGURA_JANELA, ALTURA_JANELA)
637     )
638
639     pygame.display.set_caption(
640         "RT Inspection System - Simulação"
641     )
642
643     relógio = pygame.time.Clock()
644
645     fonte_titulo = pygame.font.SysFont("arial", 26, bold=True)
646     fonte = pygame.font.SysFont("arial", 18)
647     fonte_pequena = pygame.font.SysFont("arial", 15)
648
649     cliente = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)
650
651     cliente.on_connect = ao_conectar
652     cliente.on_disconnect = ao_desconectar
653     cliente.on_message = ao_receber_mensagem
654
655     try:
656         cliente.connect(BROKER, PORTA, 60)
657         cliente.loop_start()
658
659     except OSError as erro:
660         simulacao.mensagem_status = (
661             f"Erro MQTT: {erro}. Verifique se o Mosquitto esta ativo."
662         )
663
664     executando = True
665     finalizacao_enviada = False
666     instante_finalizacao = None
667
668     acumulador_tempo = 0.0
669
670     print("Interface grafica da simulacao iniciada.")
671     print("A regioao vermelha entre 12 m e 15 m representa a falha severa.")
672
673     while executando:
674         dt_real = relógio.tick(60) / 1000.0
675         acumulador_tempo += dt_real
676
677         for evento in pygame.event.get():
678             if evento.type == pygame.QUIT:
679                 executando = False
680

```

```

681         if evento.type == pygame.KEYDOWN:
682             if evento.key == pygame.K_ESCAPE:
683                 executando = False
684
685     while (
686         acumulador_tempo >= PERIODO_SIMULACAO
687         and not simulacao.finalizado
688     ):
689         simulacao.atualizar_movimento(PERIODO_SIMULACAO)
690
691         sensor = simulacao.gerar_sensor()
692
693         if simulacao.mqtt_conectado:
694             cliente.publish(
695                 TOPICO_SENSORES,
696                 json.dumps(sensor)
697             )
698
699         acumulador_tempo -= PERIODO_SIMULACAO
700
701     if simulacao.finalizado and not finalizacao_enviada:
702         cliente.publish(
703             TOPICO_SENSORES,
704             json.dumps({"finalizado": True})
705         )
706
707         finalizacao_enviada = True
708         instante_finalizacao = time.monotonic()
709         simulacao.mensagem_status = (
710             "Percurso encerrado. Sensores finalizados."
711         )
712
713     tela.fill(COR_FUNDO)
714
715     desenhar_areas_especiais(tela)
716     desenhar_tunel(tela, simulacao)
717     desenhar_robo(tela, simulacao)
718
719     desenhar_painel(
720         tela,
721         fonte_titulo,
722         fonte,
723         fonte_pequena,
724         simulacao
725     )
726
727     desenhar_legenda(tela, fonte_pequena)
728
729     if simulacao.finalizado:
730         desenhar_tela_final(
731             tela,
732             fonte_titulo,
733             fonte,
734             simulacao
735         )
736
737         if (
738             instante_finalizacao is not None
739             and time.monotonic() - instante_finalizacao >= 4.0
740         ):
741             executando = False
742
743     pygame.display.flip()
744
745     if not finalizacao_enviada:
746         cliente.publish(
747             TOPICO_SENSORES,
748             json.dumps({"finalizado": True})
749         )
750
751     time.sleep(0.2)
752
753     cliente.loop_stop()

```

```

754     cliente.disconnect()
755
756     pygame.quit()
757
758     print("Simulacao grafica encerrada.")
759
760
761 if __name__ == "__main__":
762     main()

```

D.2 simulation_physics.py

```

1  import json
2  import math
3  import random
4  import time
5  from dataclasses import dataclass
6  from threading import Lock
7
8  import paho.mqtt.client as mqtt
9
10
11 BROKER = "localhost"
12 PORTA = 1883
13
14 TOPICO_SENSORES = "atr/sim/sensors"
15 TOPICO_ATUADORES = "atr/core/actuators"
16
17 PERIODO_SIMULACAO = 0.05      # 50 ms
18 COMPRIMENTO_TUNEL = 35.0     # metros
19 VELOCIDADE_MAXIMA = 3.0       # m/s
20
21 ACELERACAO_MAXIMA = 1.8       # m/s2
22 ATRITO = 0.35                 # resistencia proporcional a velocidade
23
24 DISTANCIA_NORMAL_TETO = 100.0 # centimetros
25 DESVIO_PADRAO_RUIDO = 0.8    # centimetros
26
27
28 @dataclass
29 class Atuadores:
30     aceleracao: int = 0
31     camera_ligada: bool = False
32
33
34 class SimulacaoTunel:
35     def __init__(self) -> None:
36         self.x = 0.0
37         self.velocidade = 0.0
38
39         self.estado_encoder = False
40         self.ultimo_metro_encoder = 0
41
42         self.atuadores = Atuadores()
43         self.mutex_atuadores = Lock()
44
45         self.camera_anterior = False
46         self.inicio = time.monotonic()
47
48     def distancia_teto(self, x: float) -> float:
49         """
50         Retorna a distancia entre o sensor LIDAR e o teto em centimetros.
51
52         Trechos normais: aproximadamente 100 cm.
53         Entre 12 m e 15 m: cavidade severa, aumentando a distancia.
54         Entre 24 m e 26 m: pequena saliencia, sem necessariamente
55         representar falha severa.
56         """
57         if 12.0 <= x <= 15.0:
58             return 128.0

```

```

59
60     if 24.0 <= x <= 26.0:
61         return 94.0
62
63     ondulacao = 1.5 * math.sin(x * 0.6)
64     return DISTANCIA_NORMAL_TETO + ondulacao
65
66 def atualizar_encoder(self) -> None:
67     metro_atual = int(self.x)
68
69     while self.ultimo_metro_encoder < metro_atual:
70         self.estado_encoder = not self.estado_encoder
71         self.ultimo_metro_encoder += 1
72
73 def atualizar_movimento(self, dt: float) -> None:
74     with self.mutex_atuadores:
75         comando_motor = self.atuadores.aceleracao
76
77         aceleracao_motor = (
78             comando_motor / 100.0
79         ) * ACELERACAO_MAXIMA
80
81         aceleracao_resultante = (
82             aceleracao_motor - ATRITO * self.velocidade
83         )
84
85         self.velocidade += aceleracao_resultante * dt
86
87         self.velocidade = max(
88             0.0,
89             min(self.velocidade, VELOCIDADE_MAXIMA)
90         )
91
92         self.x += self.velocidade * dt
93
94         self.x = min(self.x, COMPRIMENTO_TUNEL)
95
96         self.atualizar_encoder()
97
98 def gerar_sensor(self) -> dict:
99     distancia_real = self.distancia_teto(self.x)
100     ruído = random.gauss(0.0, DESVIO_PADRAO_RUIDO)
101
102     leitura_lidar = int(round(distancia_real + ruído))
103
104     timestamp = round(time.monotonic() - self.inicio, 3)
105
106     return {
107         "i_encoder": self.estado_encoder,
108         "i_lidar": leitura_lidar,
109         "velocidade": round(self.velocidade, 3),
110         "posicao_real": round(self.x, 3),
111         "timestamp": timestamp
112     }
113
114 def receber_atuadores(self, mensagem: dict) -> None:
115     if mensagem.get("finalizado", False):
116         return
117
118     aceleracao = int(mensagem.get("o_aceleracao", 0))
119     camera_ligada = bool(mensagem.get("o_liga_camera", False))
120
121     with self.mutex_atuadores:
122         self.atuadores.aceleracao = aceleracao
123         self.atuadores.camera_ligada = camera_ligada
124
125     if camera_ligada != self.camera_anterior:
126         estado = "LIGADA" if camera_ligada else "DESLIGADA"
127         print(f"[CAMERA] Camera {estado} em x={self.x:.2f} m")
128         self.camera_anterior = camera_ligada
129
130
131 simulacao = SimulacaoTunel()

```

```

132
133
134 def ao_conectar(cliente, dados_usuario, flags, codigo_motivo, propriedades):
135     print(f"Simulacao conectada ao broker MQTT: {codigo_motivo}")
136
137     cliente.subscribe(TOPICO_ATUADORES)
138
139     print(f"Assinando atuadores em: {TOPICO_ATUADORES}")
140     print(f"Publicando sensores em: {TOPICO_SENSORES}")
141
142
143 def ao_receber_mensagem(cliente, dados_usuario, mensagem):
144     try:
145         payload = mensagem.payload.decode("utf-8")
146         dados = json.loads(payload)
147
148         simulacao.receber_atuadores(dados)
149
150     except (UnicodeDecodeError, json.JSONDecodeError, ValueError) as erro:
151         print(f"Erro ao interpretar atuadores: {erro}")
152
153
154 def main() -> None:
155     cliente = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)
156
157     cliente.on_connect = ao_conectar
158     cliente.on_message = ao_receber_mensagem
159
160     cliente.connect(BROKER, PORTA, 60)
161     cliente.loop_start()
162
163     print("Simulacao fisica iniciada.")
164     print(f"Comprimento do tunel: {COMPRIMENTO_TUNEL} m")
165     print("Falha principal posicionada entre 12 m e 15 m.")
166     print()
167
168     proxima_execucao = time.monotonic()
169     contador = 0
170
171     try:
172         while simulacao.x < COMPRIMENTO_TUNEL:
173             proxima_execucao += PERIODO_SIMULACAO
174
175             simulacao.atualizar_movimento(PERIODO_SIMULACAO)
176
177             sensor = simulacao.gerar_sensor()
178
179             cliente.publish(
180                 TOPICO_SENSORES,
181                 json.dumps(sensor)
182             )
183
184             if contador % 10 == 0:
185                 with simulacao.mutex_atuadores:
186                     aceleracao = simulacao.atuadores.aceleracao
187                     camera = simulacao.atuadores.camera_ligada
188
189                 print(
190                     f"x={sensor['posicao_real']:6.2f} m | "
191                     f"v={sensor['velocidade']:4.2f} m/s | "
192                     f"lidar={sensor['i_lidar']:3d} cm | "
193                     f"encoder={int(sensor['i_encoder'])} | "
194                     f"motor={aceleracao:4d} | "
195                     f"camera={camera}"
196                 )
197
198             contador += 1
199
200             espera = proxima_execucao - time.monotonic()
201
202             if espera > 0:
203                 time.sleep(espera)
204

```

```
205 except KeyboardInterrupt:
206     print("\nSimulacao interrompida pelo usuario.")
207
208 finally:
209     cliente.publish(
210         TOPICO_SENSORES,
211         json.dumps({"finalizado": True})
212     )
213
214     time.sleep(0.3)
215
216     cliente.loop_stop()
217     cliente.disconnect()
218
219     print()
220     print("Simulacao fisica finalizada.")
221     print(f"Posicao final: {simulacao.x:.2f} m")
222     print(f"Velocidade final: {simulacao.velocidade:.2f} m/s")
223
224
225 if __name__ == "__main__":
226     main()
```

D.3 simulation_test.py

```
1 import json
2 import time
3
4 import paho.mqtt.client as mqtt
5
6
7 BROKER = "localhost"
8 PORTA = 1883
9 TOPICO_SENSORES = "atr/sim/sensors"
10
11
12 def main() -> None:
13     cliente = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)
14     cliente.connect(BROKER, PORTA, 60)
15     cliente.loop_start()
16
17     estado_encoder = False
18
19     print("Simulacao de teste iniciada.")
20
21     for indice in range(10):
22         estado_encoder = not estado_encoder
23
24         lidar = 140 if indice == 5 else 100 + indice
25
26         mensagem = {
27             "i_encoder": estado_encoder,
28             "i_lidar": lidar,
29             "timestamp": round(indice * 0.5, 2)
30         }
31
32         cliente.publish(
33             TOPICO_SENSORES,
34             json.dumps(mensagem)
35         )
36
37         print(f"Sensor publicado: {mensagem}")
38
39         time.sleep(0.5)
40
41     mensagem_final = {
42         "finalizado": True
43     }
44
45     cliente.publish(
```

```
46     TOPICO_SENSORES,
47     json.dumps(mensagem_final)
48 )
49
50 print("Mensagem de finalizacao publicada.")
51
52 time.sleep(0.5)
53
54 cliente.loop_stop()
55 cliente.disconnect()
56
57
58 if __name__ == "__main__":
59     main()
```

D.4 actuator_receiver_test.py

```
1
2 import json
3
4 import paho.mqtt.client as mqtt
5
6
7 BROKER = "localhost"
8 PORTA = 1883
9 TOPICO_ATUADORES = "atr/core/actuators"
10
11
12 def ao_conectar(cliente, dados_usuario, flags, codigo_motivo, propriedades):
13     print(f"Conectado ao broker MQTT. Resultado: {codigo_motivo}")
14     cliente.subscribe(TOPICO_ATUADORES)
15     print(f"Aguardando atuadores no topico: {TOPICO_ATUADORES}")
16
17
18 def ao_receber_mensagem(cliente, dados_usuario, mensagem):
19     payload = mensagem.payload.decode("utf-8")
20     dados = json.loads(payload)
21
22     if dados.get("finalizado", False):
23         print("Finalizacao dos atuadores recebida.")
24         cliente.disconnect()
25         return
26
27     aceleracao = dados["o_aceleracao"]
28     camera_ligada = dados["o_liga_camera"]
29
30     print(
31         f"Atuadores recebidos: "
32         f"aceleracao={aceleracao} "
33         f"camera={camera_ligada}"
34     )
35
36
37 def main() -> None:
38     cliente = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)
39
40     cliente.on_connect = ao_conectar
41     cliente.on_message = ao_receber_mensagem
42
43     cliente.connect(BROKER, PORTA, 60)
44
45     cliente.loop_forever()
46
47
48 if __name__ == "__main__":
49     main()
```


D.5 remote_gui.py

```

1 import json
2 import time
3 from dataclasses import dataclass, field
4 from threading import Lock
5
6 import paho.mqtt.client as mqtt
7 import pygame
8
9
10 # =====
11 # Configuracoes MQTT
12 # =====
13
14 BROKER = "localhost"
15 PORTA = 1883
16
17 TOPICO_COMANDOS = "atr/remote/commands"
18 TOPICO_ESTADO = "atr/core/state"
19 TOPICO_SUPERFICIE = "atr/core/surface"
20
21
22 # =====
23 # Configuracoes graficas
24 # =====
25
26 LARGURA_JANELA = 1200
27 ALTURA_JANELA = 720
28
29 COR_FUNDO = (18, 23, 31)
30 COR_PAINEL = (29, 37, 48)
31 COR_PAINEL_SECUNDARIO = (35, 44, 57)
32
33 COR_TEXTO = (234, 239, 244)
34 COR_TEXTO_SECUNDARIO = (156, 171, 186)
35
36 COR_VERDE = (66, 214, 122)
37 COR_VERMELHO = (231, 76, 76)
38 COR_AMARELO = (240, 190, 70)
39 COR_AZUL = (76, 164, 232)
40 COR_CINZA = (102, 112, 126)
41
42 COR_GRAFICO = (65, 192, 226)
43 COR_LINHA_LIMITE = (236, 106, 106)
44
45
46 # =====
47 # Estado local
48 # =====
49
50 @dataclass
51 class ComandoRemoto:
52     c_automatico: bool = True
53     c_man: bool = False
54     c_direita: bool = False
55     c_esquerda: bool = False
56     c_para: bool = False
57     j_sp_velocidade: int = 2
58     limite_falha: float = 5.0
59
60
61 @dataclass
62 class EstadoRecebido:
63     e_automatico: bool = True
64     e_inspecao: bool = False
65     posicao_x: float = 0.0
66     velocidade: float = 0.0
67     o_aceleracao: int = 0
68     o_liga_camera: bool = False
69     limite_falha: float = 5.0
70
71

```

```

72 @dataclass
73 class DadosInterface:
74     comando: ComandoRemoto = field(default_factory=ComandoRemoto)
75     estado: EstadoRecebido = field(default_factory=EstadoRecebido)
76
77     pontos_superficie: dict[float, float] = field(default_factory=dict)
78
79     conectado: bool = False
80     finalizado: bool = False
81     status: str = "Aguardando conexao MQTT..."
82
83     mutex: Lock = field(default_factory=Lock)
84
85
86 dados = DadosInterface()
87
88
89 # =====
90 # MQTT
91 # =====
92
93 def publicar_comando(cliente: mqtt.Client) -> None:
94     with dados.mutex:
95         mensagem = {
96             "c_automatico": dados.comando.c_automatico,
97             "c_man": dados.comando.c_man,
98             "c_direita": dados.comando.c_direita,
99             "c_esquerda": dados.comando.c_esquerda,
100             "c_para": dados.comando.c_para,
101             "j_sp_velocidade": dados.comando.j_sp_velocidade,
102             "limite_falha": dados.comando.limite_falha
103         }
104
105     cliente.publish(
106         TOPICO_COMANDOS,
107         json.dumps(mensagem)
108     )
109
110
111 def ao_conectar(
112     cliente: mqtt.Client,
113     dados_usuario,
114     flags,
115     codigo_motivo,
116     propriedades
117 ) -> None:
118     with dados.mutex:
119         dados.conectado = True
120         dados.status = "Interface conectada ao nucleo."
121
122     cliente.subscribe(TOPICO_ESTADO)
123     cliente.subscribe(TOPICO_SUPERFICIE)
124
125     publicar_comando(cliente)
126
127     print(f"Operacao remota conectada ao broker MQTT: {codigo_motivo}")
128     print(f"Publicando comandos em: {TOPICO_COMANDOS}")
129     print(f"Recebendo estado em: {TOPICO_ESTADO}")
130     print(f"Recebendo superficie em: {TOPICO_SUPERFICIE}")
131
132
133 def ao_desconectar(
134     cliente: mqtt.Client,
135     dados_usuario,
136     flags_desconexao,
137     codigo_motivo,
138     propriedades
139 ) -> None:
140     with dados.mutex:
141         dados.conectado = False
142         dados.status = "Conexao MQTT encerrada."
143
144

```

```

145 def ao_receber_mensagem(
146     cliente: mqtt.Client,
147     dados_usuario,
148     mensagem: mqtt.MQTTMessage
149 ) -> None:
150     try:
151         payload = mensagem.payload.decode("utf-8")
152         conteudo = json.loads(payload)
153
154         if mensagem.topic == TOPICO_ESTADO:
155             if conteudo.get("finalizado", False):
156                 with dados.mutex:
157                     dados.finalizado = True
158                     dados.status = "Execucao finalizada pelo nucleo."
159                 return
160
161             with dados.mutex:
162                 dados.estado.e_automatico = bool(
163                     conteudo.get("e_automatico", True)
164                 )
165
166                 dados.estado.e_inspecao = bool(
167                     conteudo.get("e_inspecao", False)
168                 )
169
170                 dados.estado.posicao_x = float(
171                     conteudo.get("posicao_x", 0.0)
172                 )
173
174                 dados.estado.velocidade = float(
175                     conteudo.get("velocidade", 0.0)
176                 )
177
178                 dados.estado.o_aceleracao = int(
179                     conteudo.get("o_aceleracao", 0)
180                 )
181
182                 dados.estado.o_liga_camera = bool(
183                     conteudo.get("o_liga_camera", False)
184                 )
185
186                 dados.estado.limite_falha = float(
187                     conteudo.get("limite_falha", 10.0)
188                 )
189
190             elif mensagem.topic == TOPICO_SUPERFICIE:
191                 if conteudo.get("finalizado", False):
192                     return
193
194                 x = float(conteudo.get("x", 0.0))
195                 y = float(conteudo.get("y", 0.0))
196
197                 with dados.mutex:
198                     dados.pontos_superficie[round(x, 2)] = y
199
200             except (UnicodeDecodeError, json.JSONDecodeError, ValueError) as erro:
201                 with dados.mutex:
202                     dados.status = f"Erro MQTT: {erro}"
203
204
205 # =====
206 # Desenho
207 # =====
208
209 def desenhar_texto(
210     tela: pygame.Surface,
211     fonte: pygame.font.Font,
212     texto: str,
213     posicao: tuple[int, int],
214     cor: tuple[int, int, int] = COR_TEXTO
215 ) -> None:
216     superficie = fonte.render(texto, True, cor)
217     tela.blit(superficie, posicao)

```

```
218
219
220 def desenhar_cartao(
221     tela: pygame.Surface,
222     fonte: pygame.font.Font,
223     titulo: str,
224     valor: str,
225     x: int,
226     y: int,
227     largura: int,
228     cor_valor: tuple[int, int, int] = COR_TEXTO
229 ) -> None:
230     pygame.draw.rect(
231         tela,
232         COR_PAINEL_SECUNDARIO,
233         pygame.Rect(x, y, largura, 78),
234         border_radius=8
235     )
236
237     desenhar_texto(
238         tela,
239         fonte,
240         titulo,
241         (x + 14, y + 12),
242         COR_TEXTO_SECUNDARIO
243     )
244
245     desenhar_texto(
246         tela,
247         fonte,
248         valor,
249         (x + 14, y + 43),
250         cor_valor
251     )
252
253
254 def desenhar_botao(
255     tela: pygame.Surface,
256     fonte: pygame.font.Font,
257     texto: str,
258     retangulo: pygame.Rect,
259     ativo: bool = False
260 ) -> None:
261     cor = COR_AZUL if ativo else COR_PAINEL_SECUNDARIO
262
263     pygame.draw.rect(
264         tela,
265         cor,
266         retangulo,
267         border_radius=7
268     )
269
270     largura_texto = fonte.size(texto)[0]
271     altura_texto = fonte.size(texto)[1]
272
273     desenhar_texto(
274         tela,
275         fonte,
276         texto,
277         (
278             retangulo.centerx - largura_texto // 2,
279             retangulo.centery - altura_texto // 2
280         )
281     )
282
283
284 def desenhar_grafico(
285     tela: pygame.Surface,
286     fonte: pygame.font.Font,
287     pontos: list[tuple[float, float]]
288 ) -> None:
289     area = pygame.Rect(35, 308, 765, 342)
290
```

```
291     pygame.draw.rect(  
292         tela,  
293         COR_PAINEL,  
294         area,  
295         border_radius=10  
296     )  
297  
298     desenhar_texto(  
299         tela,  
300         fonte,  
301         "Superfície reconstruída pelo LIDAR",  
302         (55, 325)  
303     )  
304  
305     grafico = pygame.Rect(65, 372, 700, 230)  
306  
307     pygame.draw.rect(  
308         tela,  
309         (21, 28, 38),  
310         grafico,  
311         border_radius=6  
312     )  
313  
314     pygame.draw.line(  
315         tela,  
316         COR_CINZA,  
317         (grafico.left, grafico.bottom),  
318         (grafico.right, grafico.bottom),  
319         2  
320     )  
321  
322     pygame.draw.line(  
323         tela,  
324         COR_CINZA,  
325         (grafico.left, grafico.top),  
326         (grafico.left, grafico.bottom),  
327         2  
328     )  
329  
330     for metro in range(0, 36, 5):  
331         x_tela = grafico.left + int((metro / 35.0) * grafico.width)  
332  
333         pygame.draw.line(  
334             tela,  
335             COR_CINZA,  
336             (x_tela, grafico.bottom),  
337             (x_tela, grafico.bottom + 6),  
338             1  
339         )  
340  
341         desenhar_texto(  
342             tela,  
343             fonte,  
344             str(metro),  
345             (x_tela - 7, grafico.bottom + 9),  
346             COR_TEXTO_SECUNDARIO  
347         )  
348  
349     for distancia in [80, 100, 120, 140]:  
350         y_tela = grafico.bottom - int(((distancia - 80) / 60.0) * grafico.height)  
351  
352         pygame.draw.line(  
353             tela,  
354             (42, 51, 64),  
355             (grafico.left, y_tela),  
356             (grafico.right, y_tela),  
357             1  
358         )  
359  
360         desenhar_texto(  
361             tela,  
362             fonte,  
363             str(distancia),
```

```

364         (grafico.left - 38, y_tela - 8),
365         COR_TEXTO_SECUNDARIO
366     )
367
368     if len(pontos) >= 2:
369         pontos_tela = []
370
371         for x, y in pontos:
372             x_limitado = max(0.0, min(x, 35.0))
373             y_limitado = max(80.0, min(y, 140.0))
374
375             x_tela = grafico.left + int(
376                 (x_limitado / 35.0) * grafico.width
377             )
378
379             y_tela = grafico.bottom - int(
380                 ((y_limitado - 80.0) / 60.0) * grafico.height
381             )
382
383             pontos_tela.append((x_tela, y_tela))
384
385         pygame.draw.lines(
386             tela,
387             COR_GRAFICO,
388             False,
389             pontos_tela,
390             3
391         )
392
393
394     def desenhar_interface(
395         tela: pygame.Surface,
396         fonte_titulo: pygame.font.Font,
397         fonte: pygame.font.Font,
398         fonte_pequena: pygame.font.Font
399     ) -> None:
400         with dados.mutex:
401             estado = EstadoRecebido(
402                 e_automatico=dados.estado.e_automatico,
403                 e_inspecao=dados.estado.e_inspecao,
404                 posicao_x=dados.estado.posicao_x,
405                 velocidade=dados.estado.velocidade,
406                 o_aceleracao=dados.estado.o_aceleracao,
407                 o_liga_camera=dados.estado.o_liga_camera,
408                 limite_falha=dados.estado.limite_falha
409             )
410
411             comando = ComandoRemoto(
412                 c_automatico=dados.comando.c_automatico,
413                 c_man=dados.comando.c_man,
414                 c_direita=dados.comando.c_direita,
415                 c_esquerda=dados.comando.c_esquerda,
416                 c_para=dados.comando.c_para,
417                 j_sp_velocidade=dados.comando.j_sp_velocidade,
418                 limite_falha=dados.comando.limite_falha
419             )
420
421         with dados.mutex:
422             pontos = sorted(dados.pontos_superficie.items())
423             conectado = dados.conectado
424             status = dados.status
425
426         tela.fill(COR_FUNDO)
427
428         pygame.draw.rect(
429             tela,
430             COR_PAINEL,
431             pygame.Rect(0, 0, LARGURA_JANELA, 72)
432         )
433
434         desenhar_texto(
435             tela,
436             fonte_titulo,
437             "RT Inspection System - Operacao Remota",

```

```

437         (28, 20)
438     )
439
440     status_mqtt = "MQTT conectado" if conectado else "MQTT desconectado"
441     cor_mqtt = COR_VERDE if conectado else COR_VERMELHO
442
443     desenhar_texto(
444         tela,
445         fonte_pequena,
446         status_mqtt,
447         (1015, 27),
448         cor_mqtt
449     )
450
451     modo = "AUTOMATICO" if estado.e_automatico else "MANUAL"
452     cor_modo = COR_VERDE if estado.e_automatico else COR_AMARELO
453
454     desenhar_cartao(
455         tela, fonte, "Modo", modo, 35, 99, 175, cor_modo
456     )
457
458     desenhar_cartao(
459         tela,
460         fonte,
461         "Posição",
462         f"{estado.posicao_x:.2f} m",
463         225,
464         99,
465         175
466     )
467
468     desenhar_cartao(
469         tela,
470         fonte,
471         "Velocidade",
472         f"{estado.velocidade:.2f} m/s",
473         415,
474         99,
475         175
476     )
477
478     cor_camera = COR_VERMELHO if estado.o_liga_camera else COR_CINZA
479
480     desenhar_cartao(
481         tela,
482         fonte,
483         "Camera",
484         "LIGADA" if estado.o_liga_camera else "DESLIGADA",
485         605,
486         99,
487         175,
488         cor_camera
489     )
490
491     cor_inspecao = COR_VERMELHO if estado.e_inspecao else COR_VERDE
492
493     desenhar_cartao(
494         tela,
495         fonte,
496         "Inspecao",
497         "ATIVA" if estado.e_inspecao else "NORMAL",
498         795,
499         99,
500         175,
501         cor_inspecao
502     )
503
504     desenhar_cartao(
505         tela,
506         fonte,
507         "Aceleracao",
508         str(estado.o_aceleracao),
509         985,

```

```

510         99,
511         175
512     )
513
514     desenhar_texto(
515         tela,
516         fonte,
517         f"Setpoint atual: {comando.j_sp_velocidade} m/s",
518         (45, 220)
519     )
520
521     desenhar_texto(
522         tela,
523         fonte,
524         f"Limite de falha: {comando.limite_falha:.1f} cm",
525         (275, 220)
526     )
527
528     desenhar_texto(
529         tela,
530         fonte_pequena,
531         status,
532         (45, 264),
533         COR_TEXTO_SECUNDARIO
534     )
535
536     desenhar_grafico(tela, fonte_pequena, pontos)
537
538     pygame.draw.rect(
539         tela,
540         COR_PAINEL,
541         pygame.Rect(830, 308, 330, 342),
542         border_radius=10
543     )
544
545     desenhar_texto(
546         tela,
547         fonte,
548         "Comandos",
549         (852, 330)
550     )
551
552     botao_automatico = pygame.Rect(852, 378, 132, 45)
553     botao_manual = pygame.Rect(1000, 378, 132, 45)
554
555     desenhar_botao(
556         tela,
557         fonte_pequena,
558         "A - Automatico",
559         botao_automatico,
560         comando.c_automatico
561     )
562
563     desenhar_botao(
564         tela,
565         fonte_pequena,
566         "M - Manual",
567         botao_manual,
568         comando.c_man
569     )
570
571     botao_esquerda = pygame.Rect(852, 445, 82, 46)
572     botao_parar = pygame.Rect(950, 445, 82, 46)
573     botao_direita = pygame.Rect(1050, 445, 82, 46)
574
575     desenhar_botao(
576         tela,
577         fonte,
578         "<",
579         botao_esquerda,
580         comando.c_esquerda
581     )
582

```



```

583     desenhar_botao(
584         tela,
585         fonte_pequena,
586         "PARAR",
587         botao_parar,
588         comando.c_para
589     )
590
591     desenhar_botao(
592         tela,
593         fonte,
594         ">",
595         botao_direita,
596         comando.c_direita
597     )
598
599     instrucoes = [
600         "A: modo automatico",
601         "M: modo manual",
602         "Setas: movimento manual",
603         "ESPAÇO: parar",
604         "+ / -: alterar setpoint",
605         "[ / ]: alterar limite",
606         "ESC: sair"
607     ]
608
609     for indice, linha in enumerate(instrucoes):
610         desenhar_texto(
611             tela,
612             fonte_pequena,
613             linha,
614             (852, 530 + indice * 17),
615             COR_TEXTO_SECUNDARIO
616         )
617
618
619     # =====
620     # Tratamento de comandos do teclado
621     # =====
622
623     def processar_tecla_pressionada(
624         tecla: int,
625         cliente: mqtt.Client
626     ) -> None:
627         with dados.mutex:
628             if tecla == pygame.K_a:
629                 dados.comando.c_automatico = True
630                 dados.comando.c_man = False
631                 dados.comando.c_direita = False
632                 dados.comando.c_esquerda = False
633                 dados.comando.c_para = False
634                 dados.status = "Modo automatico solicitado."
635
636             elif tecla == pygame.K_m:
637                 dados.comando.c_automatico = False
638                 dados.comando.c_man = True
639                 dados.comando.c_direita = False
640                 dados.comando.c_esquerda = False
641                 dados.comando.c_para = False
642                 dados.status = "Modo manual solicitado."
643
644             elif tecla == pygame.K_RIGHT:
645                 dados.comando.c_automatico = False
646                 dados.comando.c_man = True
647                 dados.comando.c_direita = True
648                 dados.comando.c_esquerda = False
649                 dados.comando.c_para = False
650                 dados.status = "Acelerando no modo manual."
651
652             elif tecla == pygame.K_LEFT:
653                 dados.comando.c_automatico = False
654                 dados.comando.c_man = True
655                 dados.comando.c_direita = False

```

```

656         dados.comando.c_esquerda = True
657         dados.comando.c_para = False
658         dados.status = "Freando no modo manual."
659
660     elif tecla == pygame.K_SPACE:
661         dados.comando.c_automatico = False
662         dados.comando.c_man = True
663         dados.comando.c_direita = False
664         dados.comando.c_esquerda = False
665         dados.comando.c_para = True
666         dados.status = "Parada manual solicitada."
667
668     elif tecla in (pygame.K_PLUS, pygame.K_KP_PLUS, pygame.K_EQUALS):
669         dados.comando.j_sp_velocidade = min(
670             3,
671             dados.comando.j_sp_velocidade + 1
672         )
673
674         dados.status = (
675             f"Setpoint alterado para "
676             f"{dados.comando.j_sp_velocidade} m/s."
677         )
678
679     elif tecla in (pygame.K_MINUS, pygame.K_KP_MINUS):
680         dados.comando.j_sp_velocidade = max(
681             0,
682             dados.comando.j_sp_velocidade - 1
683         )
684
685         dados.status = (
686             f"Setpoint alterado para "
687             f"{dados.comando.j_sp_velocidade} m/s."
688         )
689
690     elif tecla == pygame.K_LEFTBRACKET:
691         dados.comando.limite_falha = max(
692             1.0,
693             dados.comando.limite_falha - 1.0
694         )
695
696         dados.status = (
697             f"Limite de falha alterado para "
698             f"{dados.comando.limite_falha:.1f} cm."
699         )
700
701     elif tecla == pygame.K_RIGHTBRACKET:
702         dados.comando.limite_falha = min(
703             50.0,
704             dados.comando.limite_falha + 1.0
705         )
706
707         dados.status = (
708             f"Limite de falha alterado para "
709             f"{dados.comando.limite_falha:.1f} cm."
710         )
711
712     publicar_comando(cliente)
713
714
715 def processar_tecla_solta(
716     tecla: int,
717     cliente: mqtt.Client
718 ) -> None:
719     if tecla not in (pygame.K_RIGHT, pygame.K_LEFT):
720         return
721
722     with dados.mutex:
723         if dados.comando.c_man:
724             dados.comando.c_direita = False
725             dados.comando.c_esquerda = False
726             dados.comando.c_para = False
727             dados.status = "Modo manual sem comando de aceleracao."
728

```

```

729     publicar_comando(cliente)
730
731
732     # =====
733     # Programa principal
734     # =====
735
736     def main() -> None:
737         pygame.init()
738
739         tela = pygame.display.set_mode(
740             (LARGURA_JANELA, ALTURA_JANELA)
741         )
742
743         pygame.display.set_caption(
744             "RT Inspection System - Operacao Remota"
745         )
746
747         relógio = pygame.time.Clock()
748
749         fonte_titulo = pygame.font.SysFont("arial", 27, bold=True)
750         fonte = pygame.font.SysFont("arial", 18)
751         fonte_pequena = pygame.font.SysFont("arial", 15)
752
753         cliente = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)
754
755         cliente.on_connect = ao_conectar
756         cliente.on_disconnect = ao_desconectar
757         cliente.on_message = ao_receber_mensagem
758
759         try:
760             cliente.connect(BROKER, PORTA, 60)
761             cliente.loop_start()
762
763         except OSError as erro:
764             with dados.mutex:
765                 dados.status = (
766                     f"Erro MQTT: {erro}. Verifique o broker Mosquitto."
767                 )
768
769         executando = True
770
771         while executando:
772             relógio.tick(60)
773
774             for evento in pygame.event.get():
775                 if evento.type == pygame.QUIT:
776                     executando = False
777
778                 elif evento.type == pygame.KEYDOWN:
779                     if evento.key == pygame.K_ESCAPE:
780                         executando = False
781                     else:
782                         processar_tecla_pressionada(
783                             evento.key,
784                             cliente
785                         )
786
787                 elif evento.type == pygame.KEYUP:
788                     processar_tecla_solta(
789                         evento.key,
790                         cliente
791                     )
792
793             desenhar_interface(
794                 tela,
795                 fonte_titulo,
796                 fonte,
797                 fonte_pequena
798             )
799
800             pygame.display.flip()
801

```

```

802     cliente.loop_stop()
803     cliente.disconnect()
804
805     pygame.quit()
806
807
808 if __name__ == "__main__":
809     main()

```

E Benchmark Temporal

E.1 benchmark_temporal.cpp

```

1  #include "tasks.hpp"
2
3  #include <nlohmann/json.hpp>
4
5  #include <algorithm>
6  #include <chrono>
7  #include <cmath>
8  #include <fstream>
9  #include <iomanip>
10 #include <iostream>
11 #include <numeric>
12 #include <string>
13 #include <vector>
14
15 namespace {
16
17 using Clock = std::chrono::steady_clock;
18 using json = nlohmann::json;
19
20 struct Resumo {
21     double mediaUs;
22     double p95Us;
23     double maximoUs;
24 };
25
26 Resumo Resumir(std::vector<double> amostras)
27 {
28     std::sort(amostras.begin(), amostras.end());
29
30     const double soma = std::accumulate(
31         amostras.begin(), amostras.end(), 0.0
32     );
33     const std::size_t indiceP95 = static_cast<std::size_t>(
34         std::ceil(amostras.size() * 0.95)
35     ) - 1;
36
37     return {
38         soma / static_cast<double>(amostras.size()),
39         amostras[indiceP95],
40         amostras.back()
41     };
42 }
43
44 template <typename Funcao>
45 Resumo MedirPorAmostra(int rodadas, int quantidade, Funcao funcao)
46 {
47     std::vector<double> tempos;
48     tempos.reserve(static_cast<std::size_t>(rodadas));
49
50     for (int rodada = 0; rodada < rodadas; rodada++) {
51         const auto inicio = Clock::now();
52         funcao(quantidade);
53         const auto fim = Clock::now();
54
55         const double microssegundos =
56             std::chrono::duration<double, std::micro>(fim - inicio).count();

```

```

57     tempos.push_back(microssegundos / quantidade);
58 }
59
60
61     return Resumir(tempos);
62 }
63
64 Resumo MedirDistancia(int rodadas, int quantidade)
65 {
66     return MedirPorAmostra(rodadas, quantidade, [](int n) {
67         EncoderBuffer entrada;
68         PositionBuffer saida;
69         SharedRobotState estado;
70         estado.estado.velocidade = 1.0;
71
72         for (int i = 0; i < n; i++) {
73             entrada.fila_encoder.push({i % 2 == 0, i * 0.08});
74         }
75         entrada.finalizado = true;
76
77         DistanciaPercorrida(entrada, saida, estado);
78     });
79 }
80
81 Resumo MedirReconstrucao(int rodadas, int quantidade)
82 {
83     return MedirPorAmostra(rodadas, quantidade, [](int n) {
84         SensorBuffer sensores;
85         PositionBuffer posicoes;
86         SurfaceBuffer csv;
87         SurfaceBuffer mqtt;
88         SharedRobotState estado;
89         SharedActuatorData atuadores;
90         SharedSystemParameters parametros;
91         CameraEvent camera;
92
93         for (int i = 0; i < n; i++) {
94             sensores.fila_sensor.push({
95                 i % 2 == 0,
96                 100 + ((i % 200 == 100) ? 28 : 0),
97                 i * 0.08
98             });
99             posicoes.fila_posicao.push({i * 0.08, i * 0.08});
100         }
101
102         sensores.finalizado = true;
103         posicoes.finalizado = true;
104
105         ReconstrucaoSuperficie(
106             sensores,
107             posicoes,
108             csv,
109             mqtt,
110             estado,
111             atuadores,
112             parametros,
113             camera
114         );
115     });
116 }
117
118 Resumo MedirColetor(int rodadas, int quantidade)
119 {
120     return MedirPorAmostra(rodadas, quantidade, [](int n) {
121         SurfaceBuffer entrada;
122
123         for (int i = 0; i < n; i++) {
124             entrada.fila_superficie.push({
125                 i * 0.08,
126                 i * 0.08,
127                 100.0 + std::sin(i * 0.01),
128                 0.0
129             });

```

```

130     }
131     entrada.finalizado = true;
132
133     ColetorDados(entrada);
134     });
135 }
136
137 Resumo MedirJson(int rodadas, int quantidade)
138 {
139     return MedirPorAmostra(rodadas, quantidade, [](int n) {
140         for (int i = 0; i < n; i++) {
141             const json mensagem = {
142                 {"i_encoder", i % 2 == 0},
143                 {"i_lidar", 100 + i % 30},
144                 {"velocidade", 1.25},
145                 {"timestamp", i * 0.08}
146             };
147
148             const std::string payload = mensagem.dump();
149             const json recebido = json::parse(payload);
150
151             if (recebido.at("i_lidar").get<int>() < 0) {
152                 std::abort();
153             }
154         }
155     });
156 }
157
158 double MedirCamera()
159 {
160     CameraEvent evento;
161     SharedRobotState estado;
162     SharedActuatorData atuadores;
163
164     evento.falha_detectada = true;
165     evento.finalizado = true;
166     evento.x = 13.0;
167     evento.y = 128.0;
168
169     const auto inicio = Clock::now();
170     InspecaoCamera(evento, estado, atuadores);
171     const auto fim = Clock::now();
172
173     return std::chrono::duration<double, std::milli>(fim - inicio).count();
174 }
175
176 void Imprimir(const std::string &nome, const Resumo &resumo)
177 {
178     std::cout << std::left << std::setw(28) << nome
179         << std::right << std::fixed << std::setprecision(3)
180         << std::setw(12) << resumo.mediaUs
181         << std::setw(12) << resumo.p95Us
182         << std::setw(12) << resumo.maximoUs
183         << '\n';
184 }
185
186 } // namespace
187
188 int main()
189 {
190     constexpr int rodadas = 20;
191     constexpr int quantidade = 2000;
192
193     std::ofstream nulo("/dev/null");
194     std::streambuf *saidaOriginal = std::cout.rdbuf(nulo.rdbuf());
195
196     const Resumo distancia = MedirDistancia(rodadas, quantidade);
197     const Resumo reconstrucao = MedirReconstrucao(rodadas, quantidade);
198     const Resumo coletor = MedirColetor(rodadas, quantidade);
199     const Resumo json = MedirJson(rodadas, quantidade);
200     const double cameraMs = MedirCamera();
201
202     std::cout.rdbuf(saidaOriginal);

```

```
203     std::cout << "Benchmark temporal (20 rodadas, 2000 amostras)\n";
204     std::cout << std::left << std::setw(28) << "Trecho"
205         << std::right << std::setw(12) << "media(us)"
206         << std::setw(12) << "p95(us)"
207         << std::setw(12) << "max(us)" << '\n';
208
209
210     Imprimir("DistanciaPercorrida", distancia);
211     Imprimir("ReconstrucaoSuperficie", reconstrucao);
212     Imprimir("ColetorDados", coletor);
213     Imprimir("JSON dump + parse", json);
214
215     std::cout << "InspecaoCamera: " << std::fixed << std::setprecision(3)
216         << cameraMs << " ms\n";
217
218     return 0;
219 }
```